

Sustainable Local RAG Architectures for Energy Governance: A Green AI Approach via Document Slice Contextualization

Martin Tibor Sándor^{1*} and Róbert Lakatos²

^{1*}Faculty of Informatics, University of Debrecen, Debrecen, Hungary.

²Department of Data Science and Visualization, Faculty of Informatics, University of Debrecen, Debrecen, Hungary.

*Corresponding author(s). E-mail(s): sandor.martin@inf.unideb.hu;
Contributing authors: lakatos.robert@inf.unideb.hu;

Abstract

Background: Retrieval-augmented generation pipelines that contextualise every chunk against its whole document follow a “Red AI” pattern: quadratic self-attention and a linearly growing key–value cache push index-time inference beyond the 8 GB of a consumer GPU, while energy-sector auditing and policy work, bound by data-sovereignty rules, must run locally.

Results: We formalise Document Slice contextualisation, which conditions each chunk summary on a bounded window of neighbouring chunks, and add a two-tier routing agent that picks the radius per chunk with a regex gate and one single-token forward pass of the same 4B-parameter model. On 25 scientific papers (382 chunks, 250 multi-hop questions) the router chose a mean radius of 1.37, skipped 32 % of the summaries and reached Recall@20 of 0.952 against 0.954 for full-document contextualisation, with higher MRR, while cutting tracked generation energy from 106.7 to 13.2 Wh (−87.6 %), emissions from 21.8 to 2.8 g CO₂eq and generation time from 54.9 to 8.8 min. On four European Commission policy documents the fixed-radius variant stayed within 0.6 points of the baseline at 71.8 % lower energy.

Conclusion: Bounded, routed contextualisation offers a PEP-AI pathway (Precise, Explainable, Provable) to decentralised compliance auditing: reproducible metrics, inspectable per-chunk decisions and per-run energy ledgers.

Keywords: Retrieval-Augmented Generation, Green AI, Small Language Models, Edge Inference, Energy Governance, Contextual Retrieval, Carbon Footprint

1 Introduction

Large language models (LLMs) have changed how organisations search, summarise and reason over documents [1, 2], and they have concentrated capability among those who can afford data-centre-scale compute: the “compute divide” separates institutions that can train and serve frontier models from those that cannot [3–6]. Schwartz et al. named the underlying culture “Red AI”, in which accuracy is bought with exponentially growing compute, and contrasted it with “Green AI”, which reports efficiency as a first-class result [7]. The energy, carbon and water footprints of training and serving such models are documented in detail [8–13], and reporting frameworks exist to make them comparable [14–17].

Energy governance is a domain in which these costs cannot simply be externalised to a cloud provider. Regulatory auditing of grid and market operators, the transposition of the recast Energy Efficiency Directive [18], the reporting duties of the Energy Union governance framework [19] and the Commission’s plan for digitalising the energy system [20] all produce long, structured documents that decision makers must query precisely and defensibly. At the same time the General Data Protection Regulation [21] and the Artificial Intelligence Act [22] impose data-sovereignty, transparency and record-keeping obligations on automated decision support [CITATION: AI-assisted decision support in energy regulation or policy analysis]. For a public authority or a small operator, on-premises retrieval on commodity hardware is therefore a compliance property rather than a preference, and it has to fit the 8 GB of video memory of a typical workstation GPU [23].

Retrieval-augmented generation (RAG) grounds model answers in retrieved text [24–26], and production systems now combine dense retrieval [27] with lexical scoring such as BM25 [28, 29] and late-interaction re-ranking [30–33]. Their weakest link is chunking: a passage cut from its document loses the referents that make it retrievable [34–37], and layout-aware segmentation only partly repairs this [38, 39]. Context-aware representations have been proposed at embedding time [40] and, most influentially, at index time, by prepending a model-written summary of the chunk’s role before embedding [41].

Index-time contextualisation as practised industrially sends the whole document with every chunk. For a document of n chunks and L tokens this costs $(n + 1)L$ prompt tokens per document, and because self-attention scales quadratically in sequence length [42–44] the attention work grows with $\sum_i (L + \ell_i)^2$. The memory consequence is more decisive than the arithmetic: the key–value cache of a decoder grows linearly with the context window [45–47], and once weights plus cache exceed physical VRAM the runtime pages to host memory over PCIe, where throughput collapses [48]. Efficient kernels [49], sparse attention [50] and quantisation [51] lower the constants, not the dependence on L . On an 8 GB GPU the full-document design thus reproduces the Red AI regime in miniature [52–54].

Long contexts are not only expensive; they are used badly. Liu et al. showed that models retrieve information best from the beginning and the end of a prompt and degrade for the middle [55]; the effect is architectural rather than model-specific [56–58], it persists in embedding models [59] and in multimodal retrieval [60]. A summariser

handed a whole paper to describe one paragraph thus pays quadratically for context it largely ignores.

Context-expansion strategies such as sentence-window and parent-child retrieval [61, 62], recursive summary trees [63] and graph-based summarisation [64] enrich chunks with neighbouring or hierarchical context, but with variable-length inputs that are hard to bound. Adaptive systems decide per query how much retrieval to perform [65, 66] or which model to call [67, 68]. Missing is a per-chunk decision, taken at index time, of how much neighbouring context a chunk needs, under a hard bound that keeps every prompt inside a fixed memory envelope.

This article extends our conference study of Document Slice contextualisation [69] in that direction. Its contributions are: (i) a formal definition of the bounded, boundary-clamped slice and of the index-time cost it implies; (ii) a two-tier routing agent that assigns each chunk a radius $k \in \{0, 2\}$ with a deterministic table gate and a single-token forward pass of the same 4-billion-parameter model that writes the summaries, choosing a mean radius of 1.37 and skipping a third of the chunks; (iii) a complexity and memory model that explains why the full-document baseline leaves the 8 GB envelope; (iv) an evaluation on 250 multi-hop questions in which the routed pipeline reaches Recall@20 of 0.952 against 0.954 for the full-document baseline, inside the run-to-run band of the summariser, at 12.4 % of the baseline’s tracked generation energy; (v) a cross-domain check on European Commission policy documents; and (vi) a measurement protocol, including a noise-floor analysis and per-run energy ledgers, that we consider a prerequisite for Green AI claims. Section 2 presents the method and its cost model, Sect. 3 the results, Sect. 4 the operational implications and Sect. 5 concludes.

2 Methodology

The pipeline (Fig. 1) runs entirely on one workstation with a single 8 GB GPU. A PDF is parsed into layout-aware chunks (Sect. 2.1); for every chunk a routing agent decides how much neighbouring context, if any, a small language model (SLM) should see when it writes the chunk’s context summary (Sects. 2.2 and 2.3); the summary is prepended to the chunk, embedded and indexed for hybrid retrieval (Sect. 2.4). Sections 2.5–2.8 give the cost model, the evaluation metrics, the datasets and the energy accounting. Throughout, the only GPU-resident model during context generation is `gemma3:4b-it-qat` [70], a quantisation-aware-trained 4-billion-parameter model served by Ollama [71]; it acts as router and as summariser, so no second model competes for memory.

2.1 Layout-aware document parsing

Definition 1 (Chunk sequence) Let τ be the tokenizer of the embedding model and $\ell_{\max}$ a token budget. A document is represented by the ordered sequence of chunks it is segmented into,

$$D = (c_1, \dots, c_n), \quad \ell_j = |\tau(c_j)| \leq \ell_{\max} = 2000, \quad L = \sum_{j=1}^n \ell_j, \quad \text{id}(c_j) = \text{SHA256}(\tilde{c}_j), \quad (1)$$

[FIGURE INSERTION: block diagram of the end-to-end on-premise pipeline. Left to right: PDF → Docling layout analysis and HybridChunker (2000-token cap, nomic tokenizer) → chunk sequence $c_1..c_n$ → routing agent (Tier 1 regex table gate; Tier 2 single-token forward pass of gemma3:4b-it-qat) → slice builder (radius $k_i \in \{0, 2\}$, boundary-clamped) → summariser (same resident gemma3:4b-it-qat runner, num_ctx 16384) → “summary + chunk” → nomic-embed-text-v1.5 (768-d) → LanceDB (vector column + BM25 full-text index) → hybrid search → ColBERTv2 re-ranker → top-20 chunks. Draw an “8 GB VRAM envelope” box around the model stages and annotate that Ollama and the PyTorch embedder never share the GPU (sequential execution). Mark the k=0 bypass (chunk embedded without a summariser call).]

Fig. 1 End-to-end on-premise architecture. All stages run on one 8 GB consumer GPU; the routing agent and the summariser share one resident `gemma3:4b-it-qat` runner with a 16384-token context window, and the embedding model runs after the runner has been stopped. Chunks routed to $k = 0$ bypass the summariser entirely and are embedded as they are. The peak GPU memory observed over the whole pipeline was 7695 MiB.

where $\tilde{c}_j$ is the cleaned text of the chunk and L is the token length of the whole document. The identifier is a content hash, so the same chunk receives the same identifier in every experimental condition and the question sets can reference chunks without depending on any particular index.

Segmentation is performed with Docling [38, 72]: its default converter runs layout analysis, table-structure recognition and OCR (RapidOCR) on each page, and its HybridChunker [73] first splits the document along structural elements (headings, paragraphs, list items, tables, captions) and then merges adjacent elements of the same section while the token count, measured with the tokenizer of `nomic-embed-text-v1.5`, stays below $\ell_{\max} = 2000$ (`merge_peers=True`). Unlike fixed-size token windows, the resulting chunks respect the document’s hierarchy: a table, its caption and the paragraph that discusses it tend to stay together, and a chunk rarely stops mid-sentence. Each chunk text is then normalised (Unicode NFKD, typographic dashes and quotes mapped to ASCII, URLs removed, whitespace collapsed) before hashing (Fig. 2).

Two properties of the output matter for what follows. First, Docling serialises a recognised table as one line of “<row>, <column>= <value>.” cells and emits captions on their own line, so tables are detectable with a regular expression and need no model call to be recognised. Second, layout artefacts survive in the text: formula placeholders (<!-- formula-not-decoded -->) in 51 of the 382 chunks and undecodable font glyph codes (GLYPH<...>) in 93 of them. Table 1 summarises the corpus that results.

2.2 Bounded document slices

Definition 2 (Document slice) For a chunk $c_i \in D$ and a context radius $k \in \mathbb{N}_0$ the document slice is the set of chunks within k positions of c_i ,

$$\text{Slice}(c_i, k) = \{c_j \in D \mid \max(1, i - k) \leq j \leq \min(n, i + k)\}. \quad (2)$$

[FIGURE INSERTION: flow diagram of the parsing stage. Boxes: PDF page → Docling layout analysis (text blocks, tables, captions, formulas) → structural elements → HybridChunker (token-bounded merge of peers within a section, nomic tokenizer, 2 000 tokens) → cleaning (NFKD, dash/quote normalisation, URL removal, whitespace) → SHA-256 identifier → frozen chunk JSON {text, document, id}. Annotate a serialised table line (“row, column = value.”) and a formula placeholder as examples of the artefacts the router must cope with.]

Fig. 2 Layout-aware parsing and chunking. Docling’s layout models produce structural elements; the hybrid chunker merges peers within a section under the 2 000-token budget of the embedding model’s tokenizer; the cleaned text is hashed into a stable identifier. On the 25-paper corpus this yields 382 chunks with a median length of 2 090 characters, of which 42 carry a serialised table.

Table 1 Structural parsing metadata and chunk statistics of the scientific corpus (Docling HybridChunker output).

Quantity	Value	Note
Documents	25	IEEE and Nature-portfolio papers
Chunks	382	380 distinct identifiers ¹
Chunks per document (min/median/max)	8/14/33	
Chunk length, characters (min/Q1/median/mean/Q3/max)	4/878/2 090/2 766/4 111/10 260	total 1 056 773
Chunk length, tokens (min/Q1/median/mean/Q3/max)	1/180/450/666/1 021/1 999	nomic tokenizer; total 253 488; s.d. 606
Chunks per token band ($\leq 250/251\text{--}500/501\text{--}1\,000/1\,001\text{--}1\,500/1\,501\text{--}2\,000$)	119/88/78/37/60	none above the 2 000-token cap; 25 chunks at $\geq 1\,900$
Characters per token	4.17	1 056 773/253 488
Document length, characters (min/median/max)	17 760/43 324/76 161	$\approx 4.3\text{k}/10.4\text{k}/18.3\text{k}$ tokens
Chunks with formula placeholders	51	<!-- formula-not-decoded -->
Chunks with glyph codes	93	GLYPH<...>
Chunks carrying a serialised table (Tier-1 gate)	42	cells or caption pattern
Token budget/merge policy	2 000/merge peers	HybridChunker defaults otherwise

¹The 43-character chunk “The authors declare no competing interests.” occurs in three papers and therefore has one identifier; none of the three is a gold chunk.

Definition 2 refines the chunk sequence of Definition 1 (Eq. (1)). Near a document boundary the window of Eq. (2) is truncated on one side. The implementation compensates by extending the other side, so that every prompt carries the same number of chunks whenever the document is long enough (Fig. 3). With $u_i = \max(0, k - i + 1)$ chunks missing before c_i and $o_i = \max(0, i + k - n)$ chunks missing after it, the window

actually sent to the model is

$$W_i(k) = \{ j : \max(1, i-k-o_i) \leq j \leq \min(n, i+k+u_i) \}, \quad |W_i(k)| = \min(2k+1, n), \quad (3)$$

and the target chunk sits in the centre of the window except at the edges, which keeps it away from the attention-degraded middle of long prompts [55]. The summariser receives three messages: a fixed system prompt π_s (Listing 1), a user message containing the concatenated window text under the header “FULL DOCUMENT:” (the header is kept verbatim from the full-document baseline so that the only difference between the two conditions is the content), and a user message containing the chunk under “CHUNK:”. Its answer s_i , typically one or two sentences (about 200 characters), is prepended to the chunk, and the string $s_i \oplus “\n\n” \oplus c_i$ is what is embedded and indexed. The prompt length of one call, in tokens, is therefore

$$P_i(k) = s + \sum_{j \in W_i(k)} \ell_j + \ell_i \quad \text{versus} \quad P_i^{\text{full}} = s + L + \ell_i \quad (4)$$

for the full-document design, where s is the length of the system prompt (789 characters, 164 tokens). Under $\ell_{\max} = 2000$ every slice prompt is bounded by $s + (2k + 2)\ell_{\max}$, i.e. by 16 190 tokens for $k = 3$ and 12 190 for $k = 2$, regardless of the document, which is why a fixed context window of 16 384 tokens suffices for the whole corpus, whereas the full-document baseline needed 30 000 tokens and still had to page (Sect. 2.5).

Two readings of $k = 0$ occur in this article and must not be confused. In the static ablation of Sect. 3.1 “ $k = 0$ ” means that the summariser is still called, with the chunk itself as its only context (a chunk-only summary). In the routed pipeline a radius of 0 assigned by the router means that no summariser call is made at all and the chunk is embedded as it is; this is the cheaper and, as Sect. 3 shows, equally accurate option for chunks that already describe themselves.

[FIGURE INSERTION: schematic of Eq. (3) for a document of $n = 14$ chunks and $k = 2$. Three rows of 14 boxes; highlight the window for $i = 1$ (chunks 1–5, shifted right by $u_1 = 2$), $i = 7$ (chunks 5–9, centred) and $i = 14$ (chunks 10–14, shifted left by $o_{14} = 2$); mark the target chunk in each row. The conference figure [images/underflow_adjusted_doc_slice_alt.png](#) (not in the repository) shows the same mechanism and can be reused.]

Fig. 3 Boundary clamping of the slice window. When the window of radius k would start before the first chunk or end after the last one, the truncated side is compensated on the other side, so every prompt carries $\min(2k + 1, n)$ chunks and the same number of tokens up to the chunk-length bound; the target chunk stays central except at the edges.

Listing 1 System prompt of the summariser (from `Models/chunker_full.doc.Modelfile`; typographic quotes and dashes rendered in ASCII).

```
You are a summarization assistant specialized in contextual chunk analysis.

Your task is to:
- Receive a CHUNK of text and the FULL DOCUMENT it comes from.
```

```

- Output ONLY a short, concise summary of the chunk's relevance, meaning, or contribution to the
  document as a whole.
- DO NOT include introductions, explanations, or phrases like "This chunk is..." -- just return
  the clean, succinct summary.

Be objective and focus only on what the chunk contributes in the context of the full document.

Examples of good output:
- "Introduces the central thesis about climate policy reform."
- "Provides supporting evidence for the argument on economic inequality."
- "Transitions from background information to proposed methodology."

Respond only with the summary -- no preamble, no conclusion.

```

2.3 Dynamic radius routing

The static ablation of our conference study [69] showed that $k = 2$ and $k = 3$ are the best fixed radii but also that a fixed radius is wasteful in two directions: chunks that already describe themselves (abstracts, conclusions, reference lists, author and funding statements, layout debris) gain nothing from a summary, and chunks that carry a serialised table are summarised *worse* when the window is wide, because the model then describes the surrounding prose or degenerates into one of the examples of its own system prompt. We therefore let a routing agent $\mathcal{R}$ choose the radius per chunk. The agent has two tiers, both cheap, and is applied to every chunk of a document before any summary is written, so that the summariser’s request sequence is independent of the routing decisions.

2.3.1 Tier 1: deterministic table gate

A chunk is a table chunk if one of its lines contains at least three serialised cells and is dense in cells, or if a table caption starts a line:

$$\text{table}(c) = \left[\exists \text{line } \lambda \subseteq c : \# \text{cells}(\lambda) \geq 3 \wedge |\lambda|/\# \text{" = " }(\lambda) \leq 100 \right] \vee \text{caption}(c), \quad (5)$$

where a cell matches the pattern “, <col>= <val>.” and a caption matches “TABLE I”, “Table 1:”, “Table 1 |” and their variants at the start of a line. In-text mentions (“shown in Table 1”) and mathematical prose (“ $V = \{1, \dots, m\}$ ”) do not match. Table chunks receive $k_T = 2$ without a model call.

2.3.2 Tier 2: one forward pass of the SLM

Every other chunk is classified by the resident `gemma3:4b-it-qat` in a single forward pass that emits one token. The prompt consists of the router system prompt π_r (Listing 2), ten in-context exemplars $\mathcal{X}$ shown as “Passage: .../ Answer: S” pairs, and the query chunk clipped to its first 1 500 and last 500 characters. Rather than parsing the generated text, the agent reads the log-probabilities of the top-20 candidate tokens returned by the server and takes the argmax over the two answer letters, which cannot fail to parse; if a server returns no log-probabilities the emitted token is parsed, and if that fails too the paper’s default $k_{\text{default}} = 2$ is used. The letter map is $\kappa(\text{S}) = 0$

(self-describing: no summariser call) and $\kappa(N) = 2$ (needs its neighbours). The complete policy is

$$\mathcal{R}(c_i) = \begin{cases} k_{\text{fix}} & \text{if a constant radius } k_{\text{fix}} \text{ is set (control runs),} \\ k_T = 2 & \text{if } \text{table}(c_i) \text{ (Tier 1),} \\ \kappa\left(\arg \max_{\lambda \in \{S, N\}} \log p_\theta(\lambda \mid \pi_r, \mathcal{X}, \text{clip}(c_i))\right) & \text{otherwise (Tier 2),} \end{cases} \quad (6)$$

and the quantities reported for a run are the mean radius and the number of summariser calls,

$$\bar{k} = \frac{1}{n} \sum_{i=1}^n k_i, \quad N_{\text{calls}} = |\{i : k_i > 0\}|. \quad (7)$$

Algorithm 1 lists the procedure of Eq. (6); Table 2 collects the prompt templates and decision criteria, and Eq. (7) defines the two summary statistics reported for every run.

The routing pass is inexpensive. One forward pass takes about 0.16s on the RTX 4060 (median 155 ms, range 96–287 ms, with the ten exemplars forming a cached prefix of roughly 2000 tokens), so the whole 382-chunk corpus is routed in about one minute, and because the router uses the same model and the same 16384-token context window as the summariser, no model is unloaded or reloaded between the two phases. [VERIFY: latencies printed by `RnD/ollama_dynamic_slice_length.ipynb` (mean 157 ms, median 155 ms, 96–287 ms per chunk on the 119-item gold set; 382 chunks in 60 s); the router prompt averaged 2062 tokens (max 2602) in the same notebook; “Summariser calls that (re)loaded the model: 0” is printed by cell 2 of `RnD/ablation_dynamic_doc_slice_generation.ipynb`] Two design choices follow from the study reported in Sect. 3.3: the class→radius mapping matters more than the wording of the prompt (a four-class genre prompt with radii 0/1/2/3 and a two-class prompt with radii 0/2 route the benchmark almost identically once tables are gated below 3), and the exemplars must not come from the evaluation corpus. The ten exemplars are therefore taken from six papers that are not part of any benchmark (109 chunks from Lempel and Ziv 1976, SRCNN, LeCun et al. 2015, AlphaGo Zero, the Sycamore quantum-supremacy paper and a 2023 federated-learning survey, chunked with exactly the corpus recipe); with them the two-class router agrees with the collapsed gold labels of a 119-chunk hand-labelled set on 82 of the 99 chunks that reach Tier 2, and the table gate fires on all 20 gold table chunks with no false positive. [VERIFY: gold set `RnD/dynamic_slice_length/slice_radius_gold.json` (119 items, 99 with tier slm, 20 tables); letters from `RnD/dynamic_slice_length/router_experiments/results/routes_binary_heldout.json`; gold $k = 0 \rightarrow S$, $k \geq 1 \rightarrow N$; agreement 82/99 = 82.8%, confusion $S \rightarrow N$ 10, $N \rightarrow S$ 7; exemplar sources in `default_few_shots()` of `RnD/utils/dynamic_slice_prediction.py` and `RnD/split_documents_router/`] A further caveat is numerical: at temperature 0 the argmax is deterministic for given logits, but the server’s logits depend slightly on whether the shared prefix was served from its cache, so chunks whose two letters are within about 0.1–0.3 nats can flip between runs; the agent exposes the letter log-probabilities so that such near-ties can be audited, which is the “explainable” half of the pathway we argue for in Sect. 4.

Algorithm 1 Two-tier radius routing for one document (as implemented in `utils/dynamic_slice_prediction.py`)

Require: chunk sequence $D = (c_1, \dots, c_n)$; system prompt π_r ; exemplars $\mathcal{X}$; letter map κ ; k_T ; k_{default}

Ensure: radii $k_1, \dots, k_n$ and, per chunk, the tier and the letter log-probabilities

```

1: for  $i = 1$  to  $n$  do
2:   if a constant radius  $k_{\text{fix}}$  is configured then
3:      $k_i \leftarrow k_{\text{fix}}$ ; tier  $\leftarrow$  FIXED
4:   else if table( $c_i$ ) by Eq. (5) then
5:      $k_i \leftarrow k_T$ ; tier  $\leftarrow$  TABLE
6:   else
7:      $m \leftarrow [\pi_r; \{ \text{"Passage: } x / \text{Answer: } \kappa^{-1}(k_x) \}_{(x, k_x) \in \mathcal{X}}; \text{"Passage: clip}(c_i, 1500, 500) / \text{Answer:} \}]$ 
8:      $\mathcal{L} \leftarrow$  top-20 next-token log-probabilities of the SLM for  $m$  ▷
9:     num_predict=1, temperature 0
10:     $\sigma(\lambda) \leftarrow \mathcal{L}[\lambda]$  for  $\lambda \in \{S, N\}$  present in  $\mathcal{L}$ 
11:    if  $\sigma = \emptyset$  then
12:      parse the emitted token; on failure  $k_i \leftarrow k_{\text{default}}$ 
13:    else
14:       $k_i \leftarrow \kappa(\arg \max_{\lambda} \sigma(\lambda))$ 
15:    end if
16:    tier  $\leftarrow$  SLM
17:  end if
18: end for
19: return  $(k_i, \text{tier}_i, \sigma_i)_{i=1}^n$  ▷ summaries are generated afterwards, in document order, with radius  $k_i$ ;  $k_i = 0$  means no call

```

Listing 2 System prompt of the two-class routing agent (BINARY_SYSTEM_PROMPT in `utils/dynamic_slice_prediction.py`).

You are shown a passage taken from a scientific paper. Decide whether it describes itself. Answer with a single letter.

S - Self-describing. The passage already tells the reader what the paper is about or needs no such information: the abstract; a conclusion or summary of the whole paper (typically opening with 'In this paper', 'In conclusion', 'In summary', 'This paper has proposed', 'We have proposed'); or non-content matter such as a reference list, acknowledgements, funding, author biographies or contributions, competing interests, publication metadata, licence text, layout garbage. A conclusion is S even when it is full of numbers and technical terms.

N - Needs its neighbours. Any other passage: introduction, background, related work, methods, derivations, equations, experimental setup, results, discussion, figure or table captions, bullet lists, fragments.

Reply with the letter only.

2.4 Context-enriched embedding and hybrid retrieval

The enriched text of every chunk is embedded with `nomic-embed-text-v1.5` [74], a 768-dimensional open-weight encoder chosen for its long native context (the enriched chunks reach 2000 tokens plus the summary), its competitive MTEB [75] score at a size that runs unquantised on the same GPU, and its permissive licence. The encoder

Table 2 Contextual generation prompt templates and dynamic routing decision criteria.

Component	Specification
Summariser model	<code>gemma3:4b-it-qat</code> served by Ollama; options <code>temperature 0, num_ctx 16384</code> (the <code>SYSTEM</code> block and <code>PARAMETER</code> lines of <code>Models/chunker_full_doc.Modelfile</code> , passed per call so that the base model stays resident)
Summariser system prompt	Listing 1 (789 characters)
Summariser messages	user 1: “FULL DOCUMENT:” + newline + the chunks of $W_i(k)$ joined by single spaces; user 2: “CHUNK:” + newline + c_i
Embedded text	s_i + blank line + c_i for $k_i > 0$; c_i alone for $k_i = 0$ (routed)
Router model	the same resident runner and <code>num_ctx</code> ; options <code>temperature 0, num_predict 1, top_p 1; logprobs with top_logprobs 20</code>
Router system prompt	Listing 2
Router user message	ten blocks “ <code>Passage: x / Answer: S N</code> ” followed by “ <code>Passage: clip(c_i) / Answer:</code> ”
Query clipping	first 1500 and last 500 characters, joined by “ <code>[...]</code> ” when the chunk exceeds 2020 characters
Exemplars (held-out papers)	abstract (S); formula block with “where ...” (N); related work (N); dangling caption fragment (N); reference-list excerpt (S); results pointing to a figure (N); device description (N); conclusion (S); experimental setup (N); acknowledgement (S)
Decision	argmax of the letter log-probabilities; $\kappa(S) = 0$ (no summariser call), $\kappa(N) = 2$; fallback: parse the emitted letter, else $k_{\text{default}} = 2$
Tier-1 criteria	≥ 3 cells “ <code>, <col>= <val>.</code> ” on one line with ≤ 100 characters per “ <code>=</code> ”, or a caption “ <code>TABLE I</code> ” / “ <code>Table 1:</code> ” / “ <code>Table 1 </code> ” at the start of a line $\Rightarrow k_T = 2$
Ablated variant	four-class genre prompt (A: whole-paper summary or boilerplate $\rightarrow 0$, B: self-contained exposition $\rightarrow 1$, C: section-internal technical body $\rightarrow 2$, D: fragment $\rightarrow 3$), tables $\rightarrow 3$ in the first version and $\rightarrow 1$ in the confirmation runs (Sect. 3.3)

[VERIFY: Every row transcribes constants of `RnD/utlis/dynamic_slice_prediction.py` (OLLAMA_OPTIONS, TOP_LOGPROBS, QUERY_HEAD/TAIL, LETTER_TO_K, TABLE_K, _CELL_RE, _CAPTION_RE, default_few_shots()) and of cells 1–2 of `RnD/ablation_dynamic_doc_slice_generation.ipynb` (load_modelfile, message construction, SKIP_CONTEXT_FOR_K0).]

is executed in PyTorch through LanceDB’s embedding registry [76] on CUDA in 32-bit precision with mean pooling; the implementation applies neither the model’s task prefixes nor vector normalisation, and identical settings are used for every condition, so any absolute effect of these choices cancels in the comparisons.[VERIFY: registry call `get_registry().get("huggingface").create(name=..., trust_remote_code=True, device="cuda")` in cell 5 of `RnD/ablation_dynamic_doc_slice_generation.ipynb`; pooling and the absence of prefixes/normalisation in `lancedb/embeddings/transformers.py` of the installed LanceDB 0.26.1; the vector width 768 is stored in every `RnD/db/*.lance` manifest] The LanceDB table holds the enriched text, its vector, the raw chunk, the summary, the document name, the content hash and, for the routed pipeline, the radius and the tier that produced it; a BM25 full-text index is built on the enriched text and a scalar index on the hash.

Retrieval is hybrid [32, 33]: for a question q the vector leg returns the 20 nearest chunks by exact L2 search over the embeddings and the lexical leg the 20 best BM25 matches [28, 29]; the union is re-scored by a ColBERTv2 late-interaction re-ranker [30, 31] (`colbert-ir/colbertv2.0`) on the enriched text and cut to the top 20. Every system in this article, including the full-document baseline and the no-context control, is indexed and queried with exactly this procedure, so the only experimental variable is the text that was embedded. Because the GPU cannot host the Ollama runner and the PyTorch models at the same time without spilling, the pipeline runs the phases strictly in sequence (routing and summarisation, then `ollama stop`, then embedding and indexing, then retrieval); Table 3 lists the execution environment.

Table 3 Experimental hardware and software environment and embedding configuration.

Item	Specification
CPU	Intel Core i7-14650HX, 24 threads
GPU	NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB VRAM; memory bandwidth [insert nominal GB/s]
System memory	31.06 GB
Operating system / Python	Linux (zen kernels 6.18–7.2 across the measurement dates); Python 3.13 (2026 spring and summer runs) and 3.12 (router study)
Inference server	Ollama [insert server version, e.g. from <code>ollama --version</code>]; one resident runner during generation
Summariser and router model	<code>gemma3:4b-it-qat</code> (Gemma 3, 4B parameters, quantisation-aware trained int4 weights [51, 70]); layer count, KV heads and head width [verify from the model config: n_L , n_{kv} , d_h , local/global attention pattern]
Context window (<code>num_ctx</code>)	16 384 tokens for the slice and routed pipelines; 30 000 tokens for the full-document baseline (<code>Models/chunker_anthropic.Modelfile</code>)
Decoding	temperature 0; router additionally <code>num_predict 1</code> with top-20 log-probabilities
Embedding model	<code>nomic-ai/nomic-embed-text-v1.5</code> , 768-d, mean pooling, fp32 on CUDA via the LanceDB HuggingFace registry
Vector store / search	LanceDB 0.26.1; hybrid = exact-L2 vector top-20 $\cup$ BM25 top-20 $\rightarrow$ ColBERTv2 re-rank $\rightarrow$ 20
Parsing / chunking	docling 2.68.0, docling-core 2.59.0 (HybridChunker, RapidOCR); transformers 4.57.5; torch 2.9.1
Energy tracking	CodeCarbon 3.2.3 (spring/summer 2026 rows) and 3.3.1 (September 2026 rows)
Peak GPU memory	7 695 MiB over the full pipeline; generation phase $\approx$ 4.6 GB for the runner plus $\approx$ 1 GB for Docling when run concurrently

[VERIFY: CPU, GPU, RAM, OS, Python and CodeCarbon versions: columns `cpu_model`, `gpu_model`, `ram_total_size`, `os`, `python_version`, `codecarbon_version` of `RnD/emissions_data/emissions.csv` (note that rows written by CodeCarbon 3.3.1 are shifted by two columns from `region` onward relative to the 3.2.3 header; read those rows by position). Library versions: `RnD/requirements.txt`. Context windows: the two `Modelfiles` in `RnD/Models/`. Peak memory: `README.md` line 59 and the comment in cell 4 of `RnD/doc_slice_chunking.ipynb` (“4644 MB (ollama) + ... around 1 GB”).]

2.5 Computational complexity and memory analysis

Index-time contextualisation issues one summariser call per chunk, so the cost of a document is the sum of the prompt costs of Eq. (4). Writing $\bar{\ell} = L/n$ for the mean chunk length and neglecting the fixed system prompt s , the prompt-token volumes of the three designs are

$$C_{\text{full}} = \sum_{i=1}^n (L + \ell_i) = (n+1)L, \quad C_k = \sum_{i=1}^n \left(\sum_{j \in W_i(k)} \ell_j + \ell_i \right) \leq n(2k+2)\ell_{\max}, \quad C_{\text{dyn}} = \sum_{i: k_i > 0} \left(\sum_{j \in W_i(k_i)} \ell_j + \ell_i \right) \quad (8)$$

where $\mathcal{T}$ is the set of table-gated chunks and ρ_i the router prompt of chunk i (about 2000 tokens, of which the exemplar prefix is shared and served from the cache).
[VERIFY: router prompt length: mean 2062 tokens printed by RnD/ollama.dynamic.slice.length.ipynb] The full-document volume is quadratic in the number of chunks, $(n+1)L = \Theta(n^2\bar{\ell})$, whereas the bounded volume is linear, $C_k = \Theta(n\bar{\ell})$, with a constant $2k+2$ that the router lowers further by skipping chunks.

Attention makes the gap wider. For a decoder-only transformer with N_{par} non-embedding parameters, n_L layers and attention width d_{attn} , the forward pass over a prompt of P tokens costs approximately [44]

$$F(P) \approx 2N_{\text{par}} P + 2n_L d_{\text{attn}} P^2, \quad (9)$$

so the attention term of a document is $\propto \sum_i P_i^2$.

Proposition 1 (Asymptotic index-time cost) *Let every chunk satisfy $\ell_j \leq \ell_{\max}$ and let $n \geq 2k+1$. Then the attention work of full-document contextualisation grows as $\sum_i (L + \ell_i)^2 = \Theta(n^3\bar{\ell}^2)$, while that of Document Slice contextualisation with radius k is bounded by $\sum_i P_i(k)^2 \leq n(2k+2)^2\ell_{\max}^2 = \Theta(n\ell_{\max}^2)$, independent of the document length beyond n . The two designs differ by a factor $\Theta(n^2/(2k+2)^2)$ in attention work and $\Theta(n/(2k+2))$ in prompt tokens.*

Proof For every i , $L \leq L + \ell_i \leq 2L$ and $L = n\bar{\ell}$, hence $nL^2 \leq \sum_i (L + \ell_i)^2 \leq 4nL^2$, i.e. $\sum_i (L + \ell_i)^2 = \Theta(n^3\bar{\ell}^2)$. By Eq. (3) the window holds $\min(2k+1, n) = 2k+1$ chunks, so $P_i(k) \leq (2k+1)\ell_{\max} + \ell_{\max}$ and $\sum_i P_i(k)^2 \leq n(2k+2)^2\ell_{\max}^2$; the lower bound $\sum_i P_i(k)^2 \geq n\bar{\ell}^2$ follows from $P_i(k) \geq \ell_i$ and the Cauchy–Schwarz inequality, so the sum is $\Theta(n\ell_{\max}^2)$ whenever $\bar{\ell}$ and $\ell_{\max}$ are of the same order, as they are for token-bounded chunkers. The token statement follows from Eq. (8). $\square$

On the benchmark corpus the bounds are far from tight but the ordering is the same (Fig. 4): summing Eq. (4) over the 382 chunks with the actual chunk texts, the full-document design sends 18.73 M characters (≈ 4.5 M tokens) to the summariser, fixed radii $k = 1, 2, 3, 4$ send 4.25, 6.36, 8.43 and 10.32 M, and the routed pipeline sends 4.39 M (23.4% of the baseline and 69.0% of fixed $k = 2$) in 261 instead of 382 calls; the quadratic proxy $\sum_i P_i^2$ is 4.6 times smaller for $k = 3$ and 10.8 times smaller for the routed pipeline than for the baseline.
[VERIFY: re-implementation of the cost()]

formula of `RnD/dynamic.slice.length/router.experiments/scripts/cost.py` (window characters of Eq. (3) plus the chunk, summed over `RnD/split.documents/*.json`; the full-document prompt is the space-joined document plus the chunk; the routed radii are the `doc.slice.radius` fields of `RnD/preprocessed.chunks/ablation.doc.slice.radius.dynamic.json`): 18 728 428 / 4 254 674 / 6 360 511 / 8 433 157 / 10 320 971 / 4 391 194 characters; $\sum P_i^2$ ratios 4.56 and 10.81; tokens $\approx$ characters/4.17] The bound of Eq. (4) is rarely approached: the mean $k = 2$ prompt over the corpus is about 4 200 tokens and a window of median-length chunks about 2 900, against the 12 190-token bound, because the chunker fills its 2 000-token budget for only a minority of chunks (Table 1). [VERIFY: 6 360 511/382/4.17 + 190 $\approx$ 4 180 tokens (fixed- $k = 2$ prompt volume of the `cost.py` re-implementation divided by 382 calls and by 4.17 characters per token, plus the 190-token system prompt); $6 \times 450 + 190 = 2 890$ with the token median of Table 1; script: `RnD/verification/prompt.volume.and.attention.proxy.py`] The measured generation-time ratio between the baseline and the routed pipeline, 6.3 (Sect. 3.2), lies between the linear proxy (4.3) and the quadratic proxy (10.8), as expected for prompts of a few thousand tokens where the linear term of Eq. (9) still matters and where the baseline’s paging (below) adds a cost that neither proxy models.

[FIGURE INSERTION: two-panel line chart. Panel (a): index-time prompt volume per document as a function of the number of chunks n (x-axis 8–33, the range of the corpus), series: full document $(n + 1)L$ with $L = n\bar{\ell}$, fixed $k = 1, 2, 3$ from Eq. (8), and the routed pipeline with the measured skip rate (32 % of chunks, $k = 2$ otherwise); y-axis in tokens (use $\bar{\ell} = 664$). Panel (b): the quadratic attention proxy $\sum_i P_i^2$ for the same series, log y-axis. Overlay the corpus-level measured points: 4.49 M / 1.02 M / 1.53 M / 2.02 M / 1.05 M tokens (full, $k = 1$, $k = 2$, $k = 3$, routed) and the ratios 4.6 $\times$ and 10.8 $\times$. Data source: the `cost.py` re-implementation described in the text.]

Fig. 4 Theoretical index-time cost curves. Full-document contextualisation grows quadratically in the number of chunks in prompt tokens and cubically in attention work, whereas bounded slices grow linearly with a constant set by the radius; the routing agent lowers that constant further by leaving self-describing chunks un-summarised. Over the 382-chunk corpus the routed pipeline sends 23.4 % of the baseline’s prompt text and incurs about one tenth of its quadratic attention proxy; the measured 6.3-fold reduction of generation time lies between the two proxies. [VERIFY: corpus volumes and ratios as in the preceding paragraph; $6.3 = 3294.3/525.3$ from the `duration` column of the rows timestamped 2026-03-05T23:59:43 and 2026-09-25T20:06:47 in `RnD/emissions.data/emissions.csv`]

Memory decides whether the arithmetic can be done on the device at all. During generation the GPU must hold the quantised weights M_w , the activations M_{act} and the key–value (KV) cache, which for a context of P tokens occupies

$$M_{\text{KV}}(P) = 2 n_L n_{kv} d_h b P \quad (10)$$

bytes (n_{kv} key–value heads of width d_h , b bytes per element, the factor 2 for keys and values) [45, 47]. The run stays on the device only if the cache of Eq. (10) fits next to

the weights,

$$M_w + M_{KV}(P_{\text{ctx}}) + M_{\text{act}} \leq V_{\text{GPU}} \iff P_{\text{ctx}} \leq P^* = \frac{V_{\text{GPU}} - M_w - M_{\text{act}}}{2n_L n_{kv} d_h b}, \quad (11)$$

where P_{ctx} is the context window the runtime reserves, not the length of the current prompt: Ollama allocates the cache for the configured `num_ctx` when the model is loaded. With the bounded slice, $P_i(k) \leq s + (2k + 2)\ell_{\text{max}}$ (Eq. (4)) is independent of the document, so a window of 16384 tokens covers every prompt of the corpus for $k \leq 3$ and the requirement of Eq. (11) is fixed once and for all. The full-document design needs $P_{\text{ctx}} \geq s + L_{\text{max}} + \ell_{\text{max}}$, which for the longest paper of the corpus (about 18300 tokens) already exceeds 16384 and forced a 30000-token window. [VERIFY: longest document 76161 characters (sum of chunk lengths of s41586-019-1138-y.pdf.json in RnD/split_documents/), divided by 4.17 characters per token; windows from RnD/Models/chunker_anthropic.Modelfile (30000) and chunker_full_doc.Modelfile (16384)] On the 8 GB device this crossed P^* : the runtime fell back to unified memory and streamed part of the working set from host RAM over PCIe on every decoding step (Fig. 5). In the conference study we observed host-memory growth of roughly 10 GB and a resident footprint of about 5 GB for the slice pipeline; those observations were not logged and are marked for re-measurement. [unlogged observations from citds.tex, Sect. “Hardware Profiling”; no artefact in the repository] What the energy tracker did log is consistent with a bandwidth-bound baseline: its mean GPU utilisation was 46.5% over the run, against 60.6% for the fixed $k = 3$ run and 90.1% for the routed run. [VERIFY: column gpu_utilization_percent of the rows timestamped 2026-03-05T23:59:43 (46.5), 2026-03-05T22:53:27 (60.6) and 2026-09-25T20:06:47 (90.1) in RnD/emissions_data/emissions.csv; whole-machine averages sampled every second by CodeCarbon (tracking_mode = machine)] [VERIFY the paging mechanism from the Ollama server log of a baseline run (“offloaded x/y layers to GPU” and the reported KV-cache size), and re-measure resident VRAM and host RSS with nvidia-smi/psutil for the 30000- and 16384-token windows; Table 3 needs n_L , n_{kv} , d_h of Gemma 3 4B to evaluate Eq. (11) numerically.]

[FIGURE INSERTION: memory-topology diagram. Left: GPU with 8 GB VRAM containing the int4 weights (M_w), activations and the KV cache sized by `num_ctx`; right: 31 GB host RAM; between them the PCIe link. Show two stacked-bar insets: (i) slice/routed pipeline with `num_ctx` = 16384 fitting under the 8 GB line; (ii) full-document baseline with `num_ctx` = 30000 crossing the line, with the overflow mapped to host memory and an arrow labelled “per-step PCIe traffic”. Annotate the threshold P^* of Eq. (11). Values for the bar heights are [to be measured]; the qualitative layout follows the conference paper.]

Fig. 5 Unified-memory offloading model. When the reserved context window pushes weights plus KV cache beyond the device memory, the runtime maps the overflow to host memory and every decoding step crosses the PCIe bus, which shows up as low GPU utilisation and long wall-clock times rather than as an out-of-memory failure. The bounded slice keeps the reserved window, and therefore the memory footprint, independent of document length.

2.6 Evaluation metrics and statistical protocol

For a question $q \in Q$ let G_q be its set of gold supporting chunks, R_q^K the ordered list of the top- K retrieved chunks and $\text{rank}_q(c)$ the 1-based position of chunk c in that list. We report

$$\text{Recall@}K = \frac{1}{|Q|} \sum_{q \in Q} \frac{|R_q^K \cap G_q|}{|G_q|}, \quad Q_K^+ = \{q \in Q : R_q^K \cap G_q \neq \emptyset\}, \quad (12)$$

$$\text{MRR@}K = \frac{1}{|Q_K^+|} \sum_{q \in Q_K^+} \frac{1}{\min_{c \in R_q^K \cap G_q} \text{rank}_q(c)}, \quad \text{MAR@}K = \frac{1}{|Q_K^+|} \sum_{q \in Q_K^+} \frac{1}{|R_q^K \cap G_q|} \sum_{c \in R_q^K \cap G_q} \text{rank}_q(c), \quad (13)$$

for $K \in \{5, 10, 15, 20\}$. Recall is the primary metric because every question requires at least two chunks and a generator can only use what was retrieved. The mean reciprocal rank [77] reflects the position of the first relevant chunk, and the mean average rank (MAR) is our exploratory measure of attentional accessibility: it is the mean position of *all* retrieved gold chunks and tells whether the supporting evidence sits in the top part of the context that a generator will read, where the positional bias of Sect. 1 is least harmful. Two implementation details matter for comparability. First, the ranking metrics are averaged over Q_K^+ , i.e. questions with no gold chunk in the top K are excluded rather than scored zero, exactly as in the evaluation notebooks; we therefore report $|Q_K^+|$ alongside them (for the routed pipeline 233, 246, 247 and 248 of the 250 questions at $K = 5, 10, 15, 20$).[**VERIFY: “Number of N values with NaN MRR in own metrics: 17/250, 4/250, 3/250, 2/250” printed by cell 7 of RnD/benchmarking_retrievals.ipynb; the np.nanmean calls in cell 9 implement the exclusion**] Second, gold sets are compared by content hash, so a chunk retrieved from any condition’s table is matched to the question set without string comparison.

Differences between systems are tested per question. For Recall@20 we compute the paired difference of means with a bias-corrected and accelerated (BCa) bootstrap confidence interval at the 99 % level [78, 79] (SciPy [80], 9999 resamples, fixed seed), and we apply an exact McNemar test [81, 82] to the binary event “all gold chunks of the question are in the top 20”, whose discordant counts b (only the compared system succeeds) and c (only the baseline succeeds) are reported with the p -value. Because the summariser is a stochastic component even at temperature 0 (its outputs depend on the state of the server’s cache and batching), we also estimate the noise floor of a single run by regenerating one configuration and counting the questions whose top-20 list changes; the result (Sect. 3.3) is that a single run resolves differences of roughly 0.7 points of Recall@20 and 2 points of MRR@20, and we treat smaller gaps as ties.

2.7 Datasets

2.7.1 Scientific benchmark

The primary corpus consists of 25 open-access papers from IEEE and Nature-portfolio journals across engineering, physics, computing and the life sciences, chosen for their heavy use of tables, formulas and figure references, i.e. for the layout artefacts that

make chunking hard. Following the LLM-as-a-judge paradigm [83] we built a “silver standard” question set [84]: Gemini 3 Pro was given the chunk JSON of one paper at a time and instructed, in the role of a senior NLP data scientist, to write exactly ten high-complexity question–answer pairs grounded only in the provided text, each requiring at least two distinct chunk identifiers, with a plain-text gold answer and the list of supporting chunk identifiers (the abridged prompt is reproduced in the conference paper [69]). A validator script rejects questions with fewer than two supporting chunks or with identifiers that do not exist in the paper’s chunk file. The result is 250 questions with 530 gold references over 226 distinct chunks (Table 4). Because the gold labels were produced by a large proprietary model while the summaries are written by a 4-billion-parameter open model of different provenance, a shared preference for particular chunks is unlikely to be a model-level artefact; the question set nonetheless remains a silver rather than a gold standard.

2.7.2 Policy corpus

To test the method outside scientific prose and inside the target domain of regulatory text, we use four public European Commission documents on tobacco-control policy: two Commission reports to the Parliament and the Council (COM(2016) 269 on refillable electronic cigarettes and COM(2021) 249 on the application of Directive 2014/40/EU), the implementation report on the Council Recommendation on smoke-free environments (SWD(2013) 56) and the methodology report of the Independent Advisory Panel on characterising flavours (2023). They were chunked with the same recipe and questioned with the same protocol (80 questions, 164 gold references); a random 20 % subset of the questions was checked manually for semantic alignment with the supporting chunks [confirm: the manual validation of the policy questions is described in the conference paper but no record of it exists in the repository]. Although the subject matter is health rather than energy policy, the documents share the genre that energy-governance workflows deal with: directive articles, implementation tables by member state, methodology sections and annexes.

2.7.3 Held-out exemplar papers

The ten few-shot exemplars of the router come from six further papers that appear in neither benchmark (109 chunks), so that no text the router has seen as an example can be retrieved as an answer. [VERIFY: RnD/split_documents_router/: 6 files, 109 records; RnD/input_router/ holds the PDFs]

2.8 Continuous energy and carbon accounting

Every generation run is wrapped in a CodeCarbon tracker [15, 85] that samples the instantaneous power of the GPU, the CPU and the memory once per second and integrates it over the run (Eq. (14)); emissions follow from the energy through the grid

Table 4 Evaluation corpora: the scientific silver-standard benchmark and the European Commission policy set.

Quantity	Scientific benchmark	EC policy set
Documents	25 (IEEE, Nature portfolio)	4 (COM(2016) 269, COM(2021) 3)
Chunks	382	141 (14 / 32 / 35 / 60 per document)
Characters (total)	1 056 773	259 985
Median chunk length (characters)	2 090	1 206
Chunk length, tokens (mean / median)	663.6 / 450	446.8 / 246
Questions	250 (10 per document)	80 (10 / 20 / 20 / 30 per document)
Supporting chunks per question	2–4 (mean 2.12; 223 / 24 / 3 questions with 2 / 3 / 4)	2–3 (mean 2.05; 76 / 4)
Gold references	530	164
Distinct gold chunks (share of corpus)	226 (59.5%)	94 (66.7%)
Question generator	Gemini 3 Pro	Gemini 3 Pro
Manual validation	—	20% random subset [confirm]
Router exemplar pool	6 held-out papers, 109 chunks	—

[VERIFY: Chunk counts, character statistics and per-document chunk counts: read-only pass over `RnD/split_documents/*.json` and `RnD/q_and_a/Gemini/scientific_multi_chunk_control.json` and `RnD/q_and_a/Gemini/smoking.json` (count the supporting chunks); token statistics: `len(tokenizer.tokenize(text))` with the nomic tokenizer, as for Table 1, over `RnD/split_documents/*.json` (mean 663.6, median 450, min 1, max 1992, total 62 999); the 420.94 printed refers to an earlier 144-chunk version of the policy corpus; scripts: `RnD/verification/chunk_character_statistics.py` and `RnD/split_documents_router/` (6 files, 109 records).]

factor (Eq. (15)):

$$E_{\text{total}} = \int_0^T (P_{\text{GPU}}(t) + P_{\text{CPU}}(t) + P_{\text{DRAM}}(t)) dt \approx \sum_{t=1}^{\lceil T/\Delta t \rceil} (P_{\text{GPU}}^{(t)} + P_{\text{CPU}}^{(t)} + P_{\text{DRAM}}^{(t)}) \Delta t, \quad \Delta t = 1 \text{ s}, \quad (14)$$

$$\text{Emissions} = E_{\text{total}} \times \text{CI}_{\text{grid}} \times \text{PUE}, \quad (15)$$

where CI_{grid} is the operational carbon intensity of the regional grid in $\text{g CO}_2\text{eq/kWh}$ and the power-usage effectiveness is 1 for a workstation. GPU power is read from the NVIDIA management library; CPU power was read from the processor’s RAPL counters for the spring-2026 runs and, after a permission change of the operating system, estimated from CPU load and the processor’s thermal design power for the later runs; memory power is CodeCarbon’s constant estimate. The grid factor for Hungary in the tracker’s data set is $204.19 \text{ g CO}_2\text{eq/kWh}$ for the 3.2.3 releases; the 3.3.1 releases used for the router study yield $211.5\text{--}214.0 \text{ g CO}_2\text{eq/kWh}$ on the same machine. Table 5 lists the coefficients. Two consequences follow for the comparisons of Sect. 3: energy is the primary efficiency quantity and emissions are derived from it, and because the CPU measurement method changed between the baseline runs and the router runs, the GPU energy, which was measured identically throughout, is reported next to the total as the component that is comparable without qualification. [VERIFY: measurement methods: the CodeCarbon start-up messages saved in `RnD/anthropic_traditional_chunking.ipynb` (March: “Tracking Intel CPU via RAPL interface”; July: “RAPL – Permission denied ... Falling back on CPU load mode”); the grid factor is `emissions/energy-consumed` of each row of

RnD/emissions_data/emissions.csv (204.19 for the four 3.2.3 rows; 211.50–214.03 for the complete 3.3.1 rows)]

Durations are reported in two scopes that must not be mixed. The *tracked generation scope* is the interval wrapped by the energy tracker: the routing and summarisation loop over the pre-chunked documents and the final unloading of the model, without PDF parsing, embedding or indexing; all energy and emission figures refer to this scope. The *end-to-end preprocessing scope* is the wall-clock time of the complete notebook cell that also converts and chunks the PDFs with Docling; the conference paper’s headline times (71 min 34 s for the baseline, 29 min 09 s for the fixed $k = 3$ pipeline) belong to this scope and were not energy-tracked. [confirm the tracked scope of the fixed $k = 3$ energy row of 5 March 2026: the notebook version of that date (commit fe18290 of doc.slice.chunking.ipynb) appears to run the Docling conversion inside the tracked block, whereas the baseline row wraps only the summariser loop; if so, the static $k = 3$ energy in Tables 6 and 7 is an upper bound.]

Table 5 Power-measurement coefficients, per-run power draw and grid emission factors.

Quantity	Value
Sampling interval / tracking mode / PUE	1 s / whole machine / 1.0
GPU power	measured via NVML (all runs)
CPU power	RAPL counters (runs of 5 March 2026); load-based estimate with the tracker’s TDP fallback (“4 W per thread $\times$ 24 threads = 96 W”) from 19 July 2026 on constant 20.0 W estimate (31 GB installed)
Memory power	204.19 g CO ₂ eq/kWh (CodeCarbon 3.2.3 data set)
Grid carbon intensity (Hungary)	211.5–214.0 g CO ₂ eq/kWh in the 3.3.1 rows
Mean GPU power / utilisation: full-document baseline	51.5 W / 46.5 %
Mean GPU power / utilisation: static $k = 3$	55.0 W / 60.6 %
Mean GPU power / utilisation: routed pipeline	62.5 W / 90.1 %
Mean CPU power: baseline / $k = 3$ (RAPL)	46.1 W / 58.0 W
Mean CPU power: routed pipeline (estimate)	10.4 W
Mean host memory in use: baseline / $k = 3$ / routed	15.9 / 19.4 / 17.3 GB
Water footprint	not tracked (WUE 0) [add if the journal requires it]

[VERIFY: All rows are columns of RnD/emissions_data/emissions.csv: gpu_power, gpu_utilization_percent, cpu_power, ram_used_gb, ram_power, pue, tracking_mode of the rows timestamped 2026-03-05T23:59:43 (baseline), 2026-03-05T22:53:27 (static $k = 3$) and 2026-09-25T20:06:47 (routed); the TDP fallback message is in the saved CodeCarbon log of RnD/anthropic-traditional-chunking.ipynb. Rows written by CodeCarbon 3.3.1 must be read by position (two-column shift after region).]

3 Results and empirical evaluation

All metrics are those of Sect. 2.6 (Eqs. (12) and (13)) on the corpora of Sect. 2.7. Section 3.1 reports the static radius sweep, Sect. 3.2 the primary comparison, Sect. 3.3 the routing statistics and the noise floor, Sect. 3.4 the ranking dynamics, Sect. 3.5 the policy corpus and Sect. 3.6 the hardware profile.

3.1 Static radius sweep

Table 6 and Fig. 6 report the fixed-radius ablation that motivated the router. Generation time grows linearly with the radius, by roughly five minutes per unit of k over the 382 chunks (7 min 28 s at $k = 0$ to 28 min 11 s at $k = 4$), while recall saturates: $k = 2$ and $k = 3$ are the best fixed radii (Recall@20 of 0.94667 and 0.94533) and $k = 4$ falls back to the level of $k = 1$ (0.94333) although it sends 62% more prompt text than $k = 2$. [VERIFY: times: the “ $k=0 \dots k=4$ ” comment block in cell 1 of `RnD/ablation_doc_slice_generation.ipynb` (wall-clock of the generation loop, no tracker); recall values: “METRICS AT N=20” blocks of `RnD/ablation_doc_slice_retrieval.ipynb` for the tables `ablation_doc_slice_radius_0..4`; prompt-text ratio $10\,320\,971/6\,360\,511 = 1.62$ from the `cost.py` re-implementation] The full-document baseline remains the most accurate configuration (0.95400), 0.7 points above the best fixed radius, at a tracked generation time of 54 min 54 s and 106.7 Wh. Even the no-context hybrid control reaches 0.93933, so the entire effect of contextualisation on this benchmark is confined to about 1.5 points of Recall@20 [VERIFY: Table 6: $0.95400 - 0.93933 = 0.0147$]; what the ablation shows is that most of it is obtained from the immediate neighbourhood of a chunk and none from the far context, as the positional-bias literature predicts. The ranking metrics do not favour wide windows either: the best MRR@20 (0.79900) and the best MAR@20 (3.72605) among the fixed radii belong to $k = 1$, and $k = 3$ has the worst MAR of the sweep (4.01999). [VERIFY: MRR@20 and MAR@20 columns of Table 6, from the same notebook printouts] All recall differences among $k \in \{1, \dots, 4\}$ lie inside the single-run noise band established in Sect. 3.3, so the sweep identifies a plateau rather than an optimum, which is the situation in which a per-chunk decision can save cost without losing accuracy.

[FIGURE INSERTION: heat map with columns $k \in \{0, 1, 2, 3, 4\}$ and two rows. Row “generation time (min)”: 7.47, 12.13, 17.47, 22.68, 28.18 (from Table 6); row “peak VRAM (GB)”: [not measured; record `nvidia-smi` peak per k]. Optionally a third row “prompt volume (M chars)”: 2.11, 4.25, 6.36, 8.43, 10.32. Annotate each cell with its value; use a sequential colour map per row.]

Fig. 6 Cost surface of the static radius. Generation time grows linearly in k (about five minutes per unit over 382 chunks) and prompt volume grows with $2k + 2$, while every radius keeps the reserved context window, and hence the memory footprint, constant; the peak-memory row is a placeholder until it is measured per radius. [VERIFY: times as in Table 6; prompt volumes from the `cost.py` re-implementation]

Table 6 Ablation over static window radii versus full-document context and the routed pipeline (250 questions, tracked generation)

Method	Generation time	Prompt volume (M chars)	Energy (Wh)	Recall@20
Hybrid retrieval, no context	—	0	0	0.93933
Document slice $k = 0$ (chunk-only summary)	7 min 28 s ¹	2.11	[not tracked]	0.93733
Document slice $k = 1$	12 min 08 s ¹	4.25	[not tracked]	0.94333
Document slice $k = 2$	17 min 28 s ¹	6.36	[not tracked]	0.94667
Document slice $k = 3$	22 min 41 s ¹	8.43	52.52 ²	0.94533
Document slice $k = 4$	28 min 11 s ¹	10.32	[not tracked]	0.94333
Full-document context (baseline)	54 min 54 s	18.73	106.68	0.95400
Routed pipeline (this work)	8 min 45 s	4.39	13.22	0.95200

¹Wall-clock of the generation loop as recorded in the notebook; these runs were not energy-tracked.

²Energy of a separate $k = 3$ generation of 5 March 2026 (tracked, 23 min 48 s); its retrieval metrics were not evaluated, so the row come from the ablation generation. See also the scope note in Sect. 2.8.

³The conference paper printed 0.8933 for this cell; 0.78335 is the value in the evaluation notebook.

[VERIFY: Recall/MRR/MAR: “METRICS AT N=20” printouts in `RnD/ablation_doc_slice_retrieval.ipynb` (table 1, `ablation_doc_slice_radius_0..4`, `anthropic_control_table`) and in `RnD/benchmarking_retrievals.ipynb` (routed row, own vs comments of `RnD/ablation_doc_slice_generation.ipynb` for $k = 0..4$; duration column of `RnD/emissions.data/emissions.csv` (3 294.3 s, row 2026-03-05T23:59:43) and the routed run (525.3 s, row 2026-09-25T20:06:47). Energy: `energy_consumed` of the 2026-03-05T22:53:27 ($k = 3$). Prompt volumes: `cost.py` re-implementation (Sect. 2.5). Single-run noise band ± 0.7 points of Recall@20]

3.2 Primary benchmark: full-document baseline, static slice and routed pipeline

Table 7 is the central result. On the 250 multi-hop questions the routed pipeline reaches Recall@20 of 0.952 against 0.954 for the full-document baseline and 0.94533 for the static $k = 3$ slice, i.e. it is 0.2 points below the baseline and 0.7 points above the static slice, both differences inside the single-run band; at the same time it has the best MRR@20 of the three (0.79798 against 0.78635 and 0.78774) and the best MAR@20 (3.91062 against 3.93548 and 4.01999), and it leads both at $K = 5$ (0.75167 against 0.73733 and 0.74100). [VERIFY: routed and baseline values: the four “METRICS AT N=” blocks of `RnD/benchmarking_retrievals.ipynb` (OWN = `ablation_doc_slice_radius_dynamic`, BASELINE = `anthropic_control_table`); static $k = 3$: the `ablation_doc_slice_radius_3` blocks of `RnD/ablation_doc_slice_retrieval.ipynb`] The recall trajectories (Fig. 9) show where the routed pipeline gains and loses: it retrieves more gold chunks in the first five and ten positions, falls 0.7 points behind the baseline at $K = 15$ and closes to 0.2 points at $K = 20$. The cost side of the table (Figs. 7 and 8) is not inside any noise band: the routed pipeline issued 261 summariser calls instead of 382, sent 23.4 % of the baseline’s prompt text and finished the tracked generation in 8 min 45 s instead of 54 min 54 s (6.3 times faster), consuming 13.22 Wh instead of 106.68 Wh (−87.6 %; GPU energy alone −81.2 %) and emitting 2.83 instead of 21.78 g CO₂eq (−87.0 %). [VERIFY: duration, energy_consumed, gpu_energy, emissions of the rows timestamped 2026-09-25T20:06:47 and 2026-03-05T23:59:43 in `RnD/emissions.data/emissions.csv`; percentages = $1 - \text{routed}/\text{baseline}$; calls and prompt text from the routing

of `RnD/preprocessed_chunks/ablation_doc_slice_radius_dynamic.json` (261 records with non-empty context) and the `cost.py` re-implementation] Against the static $k = 3$ slice, which our conference paper proposed, the router removes a further 63 % of generation time and 75 % of energy (60 % of GPU energy) while adding 0.7 points of recall. The static slice itself already halves the baseline’s energy (−50.8 %) and time (−56.6 % in the tracked scope, −59.3 % end to end) for a 0.9-point loss of Recall@20, which is the trade-off the conference study reported; the router turns that trade-off into one with no measurable loss. [VERIFY: $1 - 52.52/106.68 = 0.508$, $1 - 1428.2/3294.3 = 0.566$ from the rows timestamped 2026-03-05T22:53:27 and 2026-03-05T23:59:43; end-to-end $1 - 1749/4294 = 0.593$ from the notebook comments “71m 34s” (`RnD/anthropic-traditional-chunking.ipynb`) and “29m 9s” (`RnD/doc_slice_chunking.ipynb`)]

The same configuration was generated twice, independently and from a freshly loaded model: the earlier run (the replicate row of Table 7) also reached Recall@20 of 0.952, with MRR@20 0.79791 and MAR@20 3.92171, in 543.7s and 13.51Wh, and the two runs differ only in the third decimal of the ranking metrics. Per-question tests were computed on the replicate, whose per-question results are stored, and are reported as provisional until they are recomputed on the run in the table: [provisional, computed from `RnD/dynamic_slice_length/router_experiments/results/res_real.binary.t2_heldout.json` against `res_anthropic_control.table.json`: paired difference of mean Recall@20 (baseline minus routed) +0.0020, 99 % BCa bootstrap interval [−0.0153, +0.0200] (9999 resamples, seed 42, SciPy 1.17); McNemar on “all gold chunks in the top 20”: 222 questions solved by both, 5 only by the routed pipeline, 6 only by the baseline, 17 by neither, exact $p = 1.00$; per question the routed pipeline is better on 6 and worse on 7. For the static $k = 3$ slice against the baseline: difference +0.0087, interval [−0.0067, +0.0260], discordant counts 3 and 6, $p = 0.51$. Recompute for the primary run once its per-question results are saved; script: `RnD/verification/retrieval_significance_tests.py`.] Both intervals contain zero at the 99 % level, and the discordant counts show that the baseline and the routed pipeline fail on almost disjoint, very small sets of questions.

[FIGURE INSERTION: grouped bar chart with three groups (full-document baseline, static $k = 3$, routed pipeline) and three bars per group on two y-axes: total tracked energy in Wh (106.68, 52.52, 13.22), GPU energy in Wh (46.78, 21.73, 8.80) and emissions in g CO₂eq (21.78, 10.72, 2.83). Annotate the reductions relative to the baseline (−50.8 %, −87.6 % total; −53.6 %, −81.2 % GPU). Data: `RnD/emissions_data/emissions.csv`, rows of 2026-03-05T23:59:43, 2026-03-05T22:53:27 and 2026-09-25T20:06:47. The conference figure `RnD/benchmarking_results/preprocessing/eco_friendliness.png` shows the first two groups.]

Fig. 7 Tracked generation energy and greenhouse-gas emissions of the three pipelines on the 25-paper corpus. The static $k = 3$ slice halves the baseline’s energy and emissions; the routed pipeline removes 87.6 % of the energy and 87.0 % of the emissions. GPU energy, which was measured identically in every run, falls by 53.6 % and 81.2 % respectively. [VERIFY: values and percentages as in Table 7; $1 - 21.73/46.78 = 0.536$]

[FIGURE INSERTION: two-panel bar chart of preprocessing runtime. Panel (a) “tracked generation scope”: 54.9, 23.8, 8.8 min for baseline, static $k = 3$, routed (rows of `emissions.csv` as in Fig. 7). Panel (b) “end-to-end incl. PDF parsing”: 71.6 and 29.2 min for baseline and static $k = 3$ (notebook comments; `RnD/benchmarking_results/preprocessing/preprocessing_time.png` plots these two as 4294 and 1749 s) and a hatched placeholder bar for the routed pipeline marked “[not measured]”. Annotate speed-ups: $2.3\times$ and $6.3\times$ in (a), $2.5\times$ in (b).]

Fig. 8 Preprocessing runtime in the two scopes of Sect. 2.8. In the tracked generation scope the static slice is 2.3 times and the routed pipeline 6.3 times faster than full-document contextualisation; in the end-to-end scope, which adds PDF parsing, the static slice is 2.5 times faster, and the routed pipeline has not yet been timed end to end.[VERIFY: $3294.3/1428.2 = 2.31$, $3294.3/525.3 = 6.27$ (duration column of `RnD/emissions_data/emissions.csv`); $4294/1749 = 2.46$ (notebook comments)]

[FIGURE INSERTION: line chart, x-axis $K \in \{5, 10, 15, 20\}$, y-axis Recall@K. Series: full-document baseline 0.73733 / 0.86500 / 0.93333 / 0.95400; routed pipeline 0.75167 / 0.86700 / 0.92600 / 0.95200; static $k = 3$ 0.74100 / 0.85900 / 0.91067 / 0.94533; optionally no-context hybrid 0.71733 / 0.84700 / 0.91800 / 0.93933. Shade a ± 0.7 -point band around the baseline at $K = 20$ to show the single-run noise floor. Data: “METRICS AT N=” blocks of `RnD/benchmarking_retrievals.ipynb` and `RnD/ablation.doc.slice.retrieval.ipynb`; the conference figures in `RnD/benchmarking_results/retrieval/recall_*.png` can be reused.]

Fig. 9 Recall trajectories. The routed pipeline leads the full-document baseline at $K = 5$ and $K = 10$, trails it by 0.7 points at $K = 15$ and by 0.2 points at $K = 20$, inside the single-run band; the static $k = 3$ slice trails at every cut-off beyond $K = 5$. [VERIFY: values listed in the placeholder; sources as in Table 7]

competing-interest statements, plus a few section stubs and short introductions.[VERIFY: inspect the records with `doc_slice_radius == 0` in `RnD/preprocessed_chunks/ablation.doc.slice.radius.dynamic.json`]

3.3.2 How the policy was reached

The routing policy of Sect. 2.3 is the end point of an ablation that we summarise here; its full record, with more than forty simulated policies, is in the repository. The first router was a four-class genre classifier (whole-paper summary or boilerplate $\rightarrow$ no summary; self-contained exposition $\rightarrow k = 1$; section-internal technical body $\rightarrow k = 2$; fragment $\rightarrow k = 3$) with tables gated to $k = 3$. Its live run scored Recall@20 of 0.9380, below fixed $k = 2$ (0.94667) and even below the no-context control[VERIFY: `RnD/router_prompt_engineering.md` Sect. 1 and the committed output of `RnD/ablation.dynamic.doc.slice.generation.ipynb / benchmarking_retrievals.ipynb` at git commit a888f5c]; a per-question diagnosis showed that every gold chunk it lost had been routed to $k = 2$ or was a table at $k = 3$, and that the wide slices had produced degenerate table summaries (for one large table the $k = 3$ summary was a verbatim copy of an example from the system prompt). Gating tables to $k = 1$ recovered part of the loss (0.9420 in two identical live runs).[VERIFY: `real_runs/tables1.json` and `tables1_run2.json` under `RnD/dynamic.slice.length/router_experiments/`] An offline simulator that

re-assembles a policy from stored summaries and replays the exact retrieval procedure then ranked the alternatives without summariser noise: $k = 3$ is never needed once tables are gated below it; tables are best at $k = 1$ or 2 and worst at 3; skipping self-describing chunks costs nothing for recall and gives the best MRR, MAR and Recall@5; narrowing technical body text to $k = 1$ costs 0.9 points; and length-only or position-only heuristics (0.9373–0.9447) cannot replace the classifier.[[VERIFY: rows `sim\_len.\*` and `sim\_pos.\*` of `results/sweep\_results.jsonl` \(Sect. 4 of `router\_prompt\_engineering.md`\)](#)] A per-question oracle that picks the best $k \in \{1, 2, 3\}$ for every question reaches 0.964, so the headroom above fixed $k = 2$ is about 1.7 points and only 17 of the 226 gold chunks are sensitive to k at all.[[VERIFY: `scripts/oracle.py` over `results/res\_ablation\_doc\_slice\_radius.1..3.json`; quoted in Sect. 3 of `router\_prompt\_engineering.md`](#)] The simplest adequate policy, the two-class “self-describing versus needs its neighbours” prompt with tables at $k = 2$, was the best offline (0.9507) and, once its exemplars were moved from benchmark papers to held-out papers, gave 0.9487 and then 0.952 in live runs.[[VERIFY: all numbers of this paragraph are in `RnD/router\_prompt\_engineering.md`: Sect. 1 \(headline table\), Sect. 2 \(diagnosis of the 0.9380 run\), Sect. 3 \(oracle 0.964, 17 of 226 gold chunks \$k\$ -sensitive\), Sect. 4 \(master table of every simulated policy, reproduced from `router\_experiments/results/sweep\_results.jsonl` by `scripts/make\_table.py`\), Sect. 4b \(round-3 modifications\) and Sect. 8 \(exemplar re-sourcing\); live-run records in `router\_experiments/real\_runs/\*.json`](#)]

3.3.3 Noise floor

Regenerating the constant- $k = 2$ configuration through the routed notebook reproduced its Recall@20 of 0.94667 exactly, yet only 26 of the 380 summaries were identical to the earlier generation, 238 of the 250 questions received a different top-20 list (5 improved, 5 worsened) and MRR@20 moved from 0.768 to 0.792.[[VERIFY: Sect. 3 of `RnD/router\_prompt\_engineering.md`; `real\_runs/fixed2.json` \(0.94667, MRR 0.79227\) versus the table `ablation\_doc\_slice\_radius\_2` \(0.94667, MRR 0.76799\); summary and top-20 comparison by `scripts/compare\_runs.py`](#)] When, instead, the very same request sequence was issued twice from a freshly loaded model, all 380 summaries and every metric were identical.[[VERIFY: `tables1` versus `tables1\_run2` in `RnD/dynamic\_slice\_length/router\_experiments/real\_runs/` \(identical metrics; Sect. 3 of the markdown report records 380/380 identical summaries\)\] The summariser’s output therefore depends deterministically on the request history \(the state of the server’s cache and batching\) rather than on a random seed, a single run resolves about 0.7 points of Recall@20 and 2 points of MRR@20, and repeating an identical run does not sample this noise. Two consequences shape our reporting: policies are ranked offline on fixed summaries and confirmed live, and the two independent live generations of the final configuration \(0.952 and 0.952\) are the evidence that its result is not a favourable draw.\[\[VERIFY: the two generations are rows 2026-09-25T16:06:28 and 2026-09-25T20:06:47 of `emissions.csv`; metrics in `real\\_runs/binary\\_t2\\_heldout.json` and in the saved output of `RnD/benchmarking\\_retrievals.ipynb`\]\(#\)\] Moving the ten exemplars from benchmark papers to held-out papers changed the router’s letter on 15% of the chunks \(mostly more skipping of front matter, acknowledgements and](#)

conclusions) but not a single question’s outcome in the offline replay, which is why we regard the exemplar source as a matter of hygiene rather than of accuracy.[[VERIFY: Sect. 8 of RnD/router_prompt_engineering.md](#); [letter agreement 287/338 between results/routes_binary.json and results/routes_binary_heldout.json via scripts/compare_routes.py](#)]

3.4 Retrieval quality and ranking dynamics

The ranking metrics explain why skipping a third of the summaries does not hurt. A summary is prepended to the chunk before embedding, so for a chunk that already states what it is about (an abstract, a conclusion) the summary only dilutes the representation; embedding such chunks raw is what lifts the routed pipeline’s Recall@5 above the baseline (0.75167 against 0.73733) and its MRR@20 to 0.79798, the highest of all configurations in this study.[[VERIFY: “METRICS AT N=5” and “N=20” blocks of RnD/benchmarking_retrievals.ipynb](#); [compare the MRR@20 column of Table 6](#)] With MAR@20 of 3.91 the gold chunks of a question sit on average at rank four of twenty, i.e. in the first fifth of the context that a downstream generator would receive, which is the primacy region of the U-shaped accessibility curve of [Fig. 10](#); the baseline (3.94) and the static slice (4.02) are marginally deeper. The number of questions for which nothing relevant is retrieved is small and similar across systems (17, 4, 3 and 2 of 250 at $K = 5, 10, 15, 20$ for the routed pipeline against 17, 6, 2 and 2 for the baseline), so the ranking averages are computed over essentially the same question sets.[[VERIFY: Recall@5, MRR, MAR and NaN counts from the printouts of RnD/benchmarking_retrievals.ipynb](#); [baseline NaN counts 17/6/2/2 from res_anthropic_control.table.json in RnD/dynamic_slice_length/router_experiments/results/](#)]

[[FIGURE INSERTION: line chart of retrieval accuracy versus the position of the relevant passage in the context, redrawn schematically after Liu et al. \[55\]](#) (U-shape: high at the first positions, minimum in the middle, partial recovery at the end; the conference paper used [images/u_curve_no_border.png](#), not in the repository). Overlay, on a 20-position axis, markers at the mean average rank of the three systems: 3.94 (full document), 4.02 (static $k = 3$), 3.91 (routed), and shade positions 1–4 as the “top 20 %” region.]

Fig. 10 Positional accessibility of the retrieved evidence. Language models use information at the beginning and end of a prompt better than in its middle; with a mean average rank of about four in a 20-chunk context, all three pipelines place the supporting chunks in the primacy region, the routed pipeline marginally earliest.[[VERIFY: MAR values from Table 7](#)]

3.5 Cross-domain robustness on European Commission policy documents

[Table 8](#) repeats the static comparison on the policy corpus. The fixed $k = 3$ slice reaches Recall@20 of 0.9625 against 0.96875 for the full-document baseline (−0.6 points), MRR@20 0.813 against 0.822 and MAR@20 3.62 against 3.52, while

its tracked generation took 252s instead of 858s (-70.6%) and consumed 5.08 instead of 18.00 Wh (-71.8% ; GPU energy -72.1%).[**VERIFY: metrics: the “METRICS AT N=”** printout for the 80-question set saved in the committed version of `RnD/benchmarking_retrievals.ipynb` of 21 September 2026 (git commit f227c29) and `RnD/benchmarking_results/tobacco/dashboard_view_recall_tobacco.png`; time and energy: rows 2026-07-19T15:01:49 (`tobacco_document_slice`) and 2026-07-19T16:44:18 (`tobacco_traditional`) of `RnD/emissions_data/emissions.csv`] The recall gap is the same size as on the scientific corpus, and the efficiency gain is larger because the policy documents are longer (up to 60 chunks, 103 000 characters), which is exactly the regime in which Proposition 1 predicts the widest gap. Regulatory text, with its numbered articles, member-state tables and annexes, is thus served by the bounded slice at least as well as scientific prose. The routing agent has not yet been run on this corpus; its column is left open, and the prompt of Listing 2, which names abstracts and conclusions, will need policy-specific exemplars (executive summaries, legal bases, annex tables) before the comparison is meaningful.

Table 8 Out-of-distribution evaluation on the European Commission policy corpus (80 multi-hop questions, 4 documents, 141 chunks).

Metric	Full document	Static $k = 3$	Routed pipeline	$\Delta k = 3$ vs full
Recall@5	0.78958	0.75833	[run]	-3.1 pt
Recall@10	0.90625	0.90000	[run]	-0.6 pt
Recall@15	0.95000	0.94375	[run]	-0.6 pt
Recall@20	0.96875	0.96250	[run]	-0.6 pt
MRR@20	0.82218	0.81296	[run]	-1.1%
MAR@20	3.51667	3.61875	[run]	$+2.9\%$
Generation time, tracked scope	14 min 18 s	4 min 12 s	[run]	-70.6%
Energy, tracked scope (Wh)	18.00	5.08	[run]	-71.8%
of which GPU (Wh)	14.53	4.05	[run]	-72.1%
Emissions (g CO ₂ eq)	3.68	1.04	[run]	-71.8%

[confirm two properties of the stored policy tables before publication: (i) the document-slice table (`tobacco_sliding_table`) was built with the raw chunk text as the embedding source, so its summaries reached only the BM25 index, whereas the baseline table embeds summary plus chunk; (ii) whether the tracked $k = 3$ run wrapped the PDF conversion. Either would make the slice column conservative.] [**VERIFY: metrics as cited in the text; energy components from the two July rows of `RnD/emissions_data/emissions.csv` (`energy_consumed`, `gpu_energy`, `emissions`, `duration`); deltas $= 1 - k3/\text{full}$ for time and energy, relative for MRR and MAR, absolute for recall]**

3.6 Hardware profiling

The memory model of Sect. 2.5 was not instrumented per run, and Fig. 11 is therefore a placeholder for the measurement we consider necessary. What the repository records is consistent with the model: a peak of 7 695 MiB of device memory over the whole pipeline, a resident footprint of about 4.6 GB for the summariser runner during slice generation, and, from the energy tracker, a mean GPU utilisation of 46.5% for the full-document

baseline against 90.1 % for the routed pipeline and a mean host-memory occupancy of 15.9 GB against 17.3 GB. [VERIFY: README.md line 59 (7 695 MiB); cell 4 comment of RnD/doc_slice_chunking.ipynb (4 644 MB); gpu_utilization_percent and ram_used_gb of the rows timestamped 2026-03-05T23:59:43 and 2026-09-25T20:06:47 of RnD/emissions_data/emissions.csv] The low average utilisation of the baseline, at a higher reserved context window and a per-call prompt of up to 18 000 tokens, is what a bandwidth-bound run looks like from the outside; the conference study additionally observed host-memory growth approaching 10 GB and a resident device footprint of about 5 GB for the slice pipeline, but those readings were taken by hand and are marked for re-measurement rather than reported as data.

[FIGURE INSERTION: 3-D surface or scatter plot. Axes: reserved context window (tokens; at least the two operating points 16 384 and 30 000), document size (chunks, 8–33 for the scientific corpus, up to 60 for the policy corpus) and resident device memory (GB) with host-memory overflow shown as a second surface or colour. Mark the 8 GB device limit as a plane. All points are [to be measured with nvidia-smi and psutil at 1-s intervals during a baseline run and a routed run]; the only firm values are the 7 695 MiB peak and the mean host memory in use of 15.9 GB (baseline) and 17.3 GB (routed) from the energy tracker.]

Fig. 11 Memory footprint versus reserved context window and document size (placeholder). The bounded slice keeps the reserved window at 16 384 tokens for every document and stays under the 8 GB device limit; the full-document baseline reserves 30 000 tokens and crosses it, paging to host memory. Measured points are to be added.

4 Discussion and future trajectories

4.1 Operational calculus

For an authority indexing thousands of pages on premises, the relevant frontier is accuracy against time and energy. On that frontier the full-document baseline buys its last 0.2 points of Recall@20 with 8.1 times the generation energy and 6.3 times the wall-clock of the routed pipeline, and the static $k = 3$ slice sits in between, at half the baseline’s energy for a 0.9-point loss. [VERIFY: $106.68/13.22 = 8.07$ and $3294.3/525.3 = 6.27$ from Table 7; $0.954 - 0.94533 = 0.0087$] Because every recall gap between the three systems lies inside the ± 0.7 -point band of a single run, whereas the energy gaps exceed any run-to-run variation by an order of magnitude, the Pareto choice is unambiguous: bounded, routed contextualisation dominates. The absolute quantities are small, but they scale with corpus size, recur at every re-index, and the baseline’s paging penalty grows with document length.

4.2 Ablation synthesis

Replacing the static heuristic with the routing agent gained 0.7 points of Recall@20 and removed 63 % of the generation time and 75 % of the energy of the $k = 3$ slice. The development path (0.938, 0.942, 0.9487, 0.952) shows that the gain came from

the class-to-radius mapping, tables never at $k = 3$ and self-describing chunks at $k = 0$, rather than from prompt wording, and that $k = 3$ is unnecessary once tables are gated. The router costs one token per chunk, about a minute per corpus and no second model. Its limits are equally clear: the oracle headroom above fixed $k = 2$ is 1.7 points and the summariser’s run-to-run variation is of the same order, so further gains must come from the summariser or from multi-run evaluation, not from finer routing classes. [VERIFY: path and headroom: Sects. 1, 3 and 8 of RnD/router_prompt_engineering.md; $0.964 - 0.94667 = 0.017$]

4.3 Architectural constraints and roadmap

The slice is intra-document: a chunk whose meaning depends on another document, an annex cited in a report or a standard referenced by an audit, receives no help. Thousand-page audit or financial filings have not been tested; Proposition 1 predicts the largest gains there, but also the largest exposure to cross-section dependencies. The natural extension is edge-to-cloud collaborative routing: the agent already exposes a confidence margin, so near-tie chunks, or questions a local generator cannot answer from the retrieved context, can be escalated selectively to a larger model while low-entropy work stays local. Next steps are a router run on the policy corpus with policy-specific exemplars, per-run memory instrumentation, and enough repeated generations to replace single-run bands by interval estimates.

5 Conclusion

Index-time contextual retrieval can be made to fit the memory, time and energy budget of a single consumer GPU without giving up its accuracy. Bounding each chunk’s context to a window of neighbours makes prompt length and reserved memory independent of document length, and a one-token routing decision of the same small model removes a third of the summaries and most of the remaining cost. On 250 multi-hop questions over 25 scientific papers the routed pipeline matched full-document contextualisation within the noise of a single run (Recall@20 0.952 against 0.954) with better ranking metrics, at 12 % of its generation energy and 16 % of its generation time, and the fixed-radius variant transferred to European Commission policy documents with the same accuracy gap and a 72 % energy saving. [VERIFY: $13.22/106.68 = 0.124$ and $525.3/3294.3 = 0.159$ (Table 7); $1 - 5.08/18.00 = 0.718$ (Table 8)] The pipeline is precise (its metrics are reproducible per question), explainable (every routing decision is logged with its evidence) and provable (every run carries an energy and emission ledger), which is the PEP-AI footing we consider necessary for AI-assisted compliance auditing at the edge of the energy system.

Acknowledgements. An earlier version of this work was presented at the Student Research Conference of the University of Debrecen and published as a conference paper [69]. It was supported by the UD Talent Program and by the Doctoral School of Informatics and the Faculty of Informatics of the University of Debrecen.

The authors acknowledge the use of generative artificial intelligence tools, specifically Gemini (Google), Claude (Anthropic), ChatGPT (OpenAI) and Grammarly, in

preparing this manuscript and the associated research. These systems were used to improve the stylistic quality, grammatical accuracy and coherence of the text; Gemini assisted the initial literature survey and generated the silver-standard question sets under the protocol of Sect. 2.7; Gemini and ChatGPT were used as coding assistants; and Claude was used to draft this journal version from the conference manuscript and the repository’s experimental records, including the bibliography entries marked as machine-added in the source file, and to run the dynamic-radius routing experiments under the authors’ direction. The core concepts, research methodology and scientific contributions are the authors’ original work; all AI-generated outputs, including text, source code and the experimental protocol, were produced under human supervision, were validated manually and were critically reviewed and edited by the authors, who remain fully responsible for the accuracy, integrity and originality of the content.

6 Declarations

- **Funding:** UD Talent Program; Doctoral School of Informatics and Faculty of Informatics, University of Debrecen [confirm grant identifiers, if any].
- **Conflict of interest:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** The frozen chunk files, question sets, per-run energy records (emissions.csv), routing records and per-question retrieval results analysed in this article are available in the project repository [69].
- **Materials availability:** Not applicable.
- **Code availability:** The complete pipeline (parsing, routing agent, slice generation, indexing, evaluation and the offline policy simulator) is available in the same repository [69].
- **Author contribution:** [insert CRediT statement].

References

- [1] Bengio, Y., Ducharme, R., Vincent, P., Jauvin, C.: A neural probabilistic language model. *Journal of Machine Learning Research* **3**(Feb), 1137–1155 (2003)
- [2] Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., Chen, H., Liu, Z., Dou, Z., Wen, J.-R.: Large language models for information retrieval: A survey. *ACM Transactions on Information Systems* **44**(1), 1–54 (2025) <https://doi.org/10.1145/3748304>
- [3] Besiroglu, T., Bergerson, S.A., Michael, A., Heim, L., Luo, X., Thompson, N.: The Compute Divide in Machine Learning: A Threat to Academic Contribution and Scrutiny? <https://arxiv.org/abs/2401.02452> (Retrieved: 2026-01-24) (2024)
- [4] Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., Villalobos, P.: Compute trends across three eras of machine learning. In: 2022 International Joint Conference

- on Neural Networks (IJCNN), pp. 1–8. IEEE, ??? (2022). <https://doi.org/10.1109/IJCNN55064.2022.9891914>
- [5] Thompson, N.C., Greenewald, K., Lee, K., Manso, G.F.: The Computational Limits of Deep Learning (2020)
 - [6] Hooker, S.: The hardware lottery. *Communications of the ACM* **64**(12), 58–65 (2021) <https://doi.org/10.1145/3467017>
 - [7] Schwartz, R., Dodge, J., Smith, N.A., Etzioni, O.: Green ai. *Communications of the ACM* **63**(12), 54–63 (2020)
 - [8] Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: Korhonen, A., Traum, D., Màrquez, L. (eds.) *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650. Association for Computational Linguistics, Florence, Italy (2019). <https://doi.org/10.18653/v1/P19-1355> . <https://aclanthology.org/P19-1355/>
 - [9] Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., Dean, J.: Carbon Emissions and Large Neural Network Training (2021)
 - [10] Luccioni, S., Jernite, Y., Strubell, E.: Power hungry processing: Watts driving the cost of ai deployment? In: *The 2024 ACM Conference on Fairness, Accountability and Transparency. FAccT '24*, pp. 85–99. ACM, ??? (2024). <https://doi.org/10.1145/3630106.3658542> . <http://dx.doi.org/10.1145/3630106.3658542>
 - [11] Li, P., Yang, J., Islam, M.A., Ren, S.: Making AI Less "Thirsty": Uncovering and Addressing the Secret Water Footprint of AI Models (2025). <https://arxiv.org/abs/2304.03271>
 - [12] Vries, A.: The growing energy footprint of artificial intelligence. *Joule* **7**(10), 2191–2194 (2023) <https://doi.org/10.1016/j.joule.2023.09.004>
 - [13] Wu, C.-J., Raghavendra, R., Gupta, U., Acun, B., Ardalani, N., Maeng, K., Chang, G., Aga, F., Huang, J., Bai, C., *et al.*: Sustainable AI: Environmental implications, challenges and opportunities. In: *Proceedings of Machine Learning and Systems 4 (MLSys 2022)* (2022). arXiv:2111.00364
 - [14] Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., Pineau, J.: Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research* **21**(248), 1–43 (2020)
 - [15] Lacoste, A., Luccioni, A., Schmidt, V., Dandres, T.: Quantifying the Carbon Emissions of Machine Learning (2019). <https://arxiv.org/abs/1910.09700>
 - [16] Lannelongue, L., Grealey, J., Inouye, M.: Green algorithms: Quantifying the

- carbon footprint of computation. *Advanced Science* **8**(12), 2100707 (2021) <https://doi.org/10.1002/adv.202100707>
- [17] Tabbakh, A., Al Amin, L., Islam, M., Mahmud, G.M.I., Chowdhury, I., Mukta, M.: Towards sustainable ai: a comprehensive framework for green ai. *Discover Sustainability* **5** (2024) <https://doi.org/10.1007/s43621-024-00641-4>
 - [18] European Parliament and Council of the European Union: Directive (EU) 2023/1791 of 13 September 2023 on energy efficiency and amending Regulation (EU) 2023/955 (recast). *Official Journal of the European Union*, L 231, 20 September 2023, pp. 1–111 (2023)
 - [19] European Parliament and Council of the European Union: Regulation (EU) 2018/1999 of 11 December 2018 on the Governance of the Energy Union and Climate Action. *Official Journal of the European Union*, L 328, 21 December 2018, pp. 1–77 (2018)
 - [20] European Commission: Digitalising the energy system – EU action plan. Communication COM(2022) 552 final, Brussels, 18 October 2022 (2022)
 - [21] European Parliament and Council of the European Union: Regulation (EU) 2016/679 of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 4 May 2016, pp. 1–88 (2016)
 - [22] European Parliament and Council of the European Union: Regulation (EU) 2024/1689 of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, L, 12 July 2024 (2024)
 - [23] Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. *Proceedings of the IEEE* **107**(8), 1738–1762 (2019) <https://doi.org/10.1109/JPROC.2019.2918951>
 - [24] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. <https://arxiv.org/abs/2005.11401> (Retrieved: Dec. 27, 2025) (2021)
 - [25] Vinayan Kozhipuram, A., Shailendra, S., Kadel, R.: Retrieval-augmented generation vs. baseline llms: A multi-metric evaluation for knowledge-intensive content. *Information* **16**(9) (2025)
 - [26] Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K., Shoham, Y.: In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics* **11**, 1316–1331 (2023) <https://doi.org/10.1162/tacl-2023-0075>

[//doi.org/10.1162/tacl.a.00605](https://doi.org/10.1162/tacl.a.00605)

- [27] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.-t.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020), pp. 6769–6781 (2020). <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- [28] Robertson, S.E., Walker, S., Jones, S., Hancock-Beaulieu, M.M., Gatford, M.: Okapi at TREC-3. In: Proceedings of the Third Text REtrieval Conference (TREC-3), pp. 109–126. NIST Special Publication 500-225, ??? (1995)
- [29] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* **3**(4), 333–389 (2009) <https://doi.org/10.1561/15000000019>
- [30] Khattab, O., Zaharia, M.: ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2020), pp. 39–48. ACM, ??? (2020). <https://doi.org/10.1145/3397271.3401075>
- [31] Santhanam, K., Khattab, O., Saad-Falcon, J., Potts, C., Zaharia, M.: ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. <https://arxiv.org/abs/2112.01488> (Retrieved 2026-01-02) (2022)
- [32] Kiran, S.: Hybrid retrieval-augmented generation (rag) systems with embedding vector databases. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* **11**, 2694–2702 (2025) <https://doi.org/10.32628/CSEIT25112702>
- [33] Wang, X., Wang, Z., Gao, X., Zhang, F., Wu, Y., Xu, Z., Shi, T., Wang, Z., Li, S., Qian, Q., Yin, R., Lv, C., Zheng, X., Huang, X.: Searching for best practices in retrieval-augmented generation. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024) (2024). arXiv:2407.01219
- [34] Merola, C., Singh, J.: Reconstructing Context: Evaluating Advanced Chunking Strategies for Retrieval-Augmented Generation. <https://arxiv.org/abs/2504.19754> (Retrieved 2026-01-02) (2025)
- [35] Smith, B., Troynikov, A.: Evaluating chunking strategies for retrieval. Technical report, Chroma (July 2024)
- [36] Qu, R., Tu, R., Bao, F.: Is Semantic Chunking Worth the Computational Cost? <https://arxiv.org/abs/2410.13070> (Retrieved: 2026-01-24) (2024)

- [37] Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., Yu, D.: Dense X Retrieval: What Retrieval Granularity Should We Use? (2023)
- [38] Auer, C., Lysak, M., Nassar, A., Dolfi, M., Livathinos, N., Vagenas, P., Ramis, C.B., Omenetti, M., Lindlbauer, F., Dinkla, K., Mishra, L., Kim, Y., Gupta, S., Lima, R.T., Weber, V., Morin, L., Meijer, I., Kuropiatnyk, V., Staar, P.W.J.: Docling Technical Report. <https://arxiv.org/abs/2408.09869> (Retrieved 2026-01-02) (2024)
- [39] Verma, P.: S2 Chunking: A Hybrid Framework for Document Segmentation Through Integrated Spatial and Semantic Analysis. <https://arxiv.org/abs/2501.05485> (Retrieved: 2026-01-24) (2025)
- [40] Günther, M., Mohr, I., Williams, D.J., Wang, B., Xiao, H.: Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models (2024)
- [41] Anthropic: Introducing Contextual Retrieval. <https://www.anthropic.com/news/contextual-retrieval> (Retrieved: 2025-12-31) (2024)
- [42] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems 30 (NeurIPS 2017), pp. 5998–6008 (2017). arXiv:1706.03762
- [43] Tay, Y., Dehghani, M., Bahri, D., Metzler, D.: Efficient transformers: A survey. *ACM Computing Surveys* **55**(6), 1–28 (2022)
- [44] Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling Laws for Neural Language Models (2020)
- [45] Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., Dean, J.: Efficiently scaling transformer inference. In: Proceedings of Machine Learning and Systems 5 (MLSys 2023) (2023). arXiv:2211.05102
- [46] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with PagedAttention. In: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023). ACM, ??? (2023). <https://doi.org/10.1145/3600006.3613165> . arXiv:2309.06180
- [47] Ainslie, J., Lee-Thorp, J., Jong, M., Zemlyanskiy, Y., Lebrón, F., Sanghai, S.: GQA: Training generalized multi-query transformer models from multi-head checkpoints. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023) (2023). arXiv:2305.13245
- [48] Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., Zhang, C.: FlexGen: High-throughput generative inference of large language models with a single GPU. In: Proceedings of the 40th International

- Conference on Machine Learning (ICML 2023) (2023). arXiv:2303.06865
- [49] Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Ré, C.: FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In: Advances in Neural Information Processing Systems 35 (NeurIPS 2022) (2022). arXiv:2205.14135
 - [50] Beltagy, I., Peters, M.E., Cohan, A.: Longformer: The Long-Document Transformer (2020)
 - [51] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018), pp. 2704–2713 (2018). <https://doi.org/10.1109/CVPR.2018.00286>
 - [52] Shen, M., Umar, M., Maeng, K., Suh, G.E., Gupta, U.: Towards Understanding Systems Trade-offs in Retrieval-Augmented Generation Model Inference. <https://arxiv.org/abs/2412.11854> (Retrieved: 2025-12-31) (2024)
 - [53] Zhao, S., Shao, Y., Huang, Y., Song, J., Wang, Z., Wan, C., Ma, L.: Understanding the Design Decisions of Retrieval-Augmented Generation Systems. <https://arxiv.org/abs/2411.19463> (Retrieved: 2025-12-31) (2025)
 - [54] Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M., Bergeron, W., Kepner, J., Tiwari, D., Gadepally, V.: From words to watts: Benchmarking the energy costs of large language model inference. In: 2023 IEEE High Performance Extreme Computing Conference (HPEC), pp. 1–9. IEEE, ??? (2023). <https://doi.org/10.1109/HPEC58863.2023.10363447> . arXiv:2310.03003
 - [55] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the Middle: How Language Models Use Long Contexts. <https://arxiv.org/abs/2307.03172> (Retrieved 2026-01-02) (2023)
 - [56] Guo, X., Vosoughi, S.: Serial Position Effects of Large Language Models. <https://arxiv.org/abs/2406.15981> (Retrieved 2026-01-02) (2024)
 - [57] Menschikov, M., Kharitonov, A., Kotyga, M., Porvatov, V., Zhukovskaya, A., Kagramanyan, D., Shvetsov, E., Burnaev, E.: Beyond Early-Token Bias: Model-Specific and Language-Specific Position Effects in Multilingual LLMs. <https://arxiv.org/abs/2505.16134> (Retrieved 2026-01-02) (2025)
 - [58] Hsieh, C.-Y., Chuang, Y.-S., Li, C.-L., Wang, Z., Le, L.T., Kumar, A., Glass, J., Ratner, A., Lee, C.-Y., Krishna, R., Pfister, T.: Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization (2024)
 - [59] Lee, R.J., Goel, S., Ramchandran, K.: Quantifying Positional Biases in Text Embedding Models. <https://arxiv.org/abs/2412.15241> (Retrieved 2026-01-02)

(2025)

- [60] Yao, J., Liu, S., Wang, Y., Mei, L., Bi, B., Ge, Y., Li, Z., Cheng, X.: Who is in the Spotlight: The Hidden Bias Undermining Multimodal Retrieval-Augmented Generation. <https://arxiv.org/abs/2506.11063> (Retrieved: 2026-01-24) (2025)
- [61] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H.: Retrieval-Augmented Generation for Large Language Models: A Survey (2024). <https://arxiv.org/abs/2312.10997>
- [62] Liu, J.: LlamaIndex. <https://doi.org/10.5281/zenodo.1234> . https://github.com/jerryjliu/llama_index
- [63] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval (2024). <https://arxiv.org/abs/2401.18059>
- [64] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization (2024)
- [65] Jeong, S., Baek, J., Cho, S., Hwang, S.J., Park, J.C.: Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2024) (2024). arXiv:2403.14403
- [66] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: The Twelfth International Conference on Learning Representations (ICLR 2024) (2024). arXiv:2310.11511
- [67] Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J.E., Kadous, M.W., Stoica, I.: RouteLLM: Learning to Route LLMs with Preference Data (2024)
- [68] Chen, L., Zaharia, M., Zou, J.: FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance (2023)
- [69] Sándor, M.T.: Document Slice Contextual Retrieval. GitHub. Accessed: September 29, 2026(2025)
- [70] Gemma Team, Google DeepMind: Gemma 3 Technical Report (2025). <https://arxiv.org/abs/2503.19786>
- [71] Ollama: Ollama: Get up and running with large language models locally. <https://ollama.com/>. Accessed: Apr. 18, 2026 (2024)
- [72] Livathinos, N., Auer, C., Nassar, A., Lima, R., Lysak, M., Ebouky, B.,

- Berrosapi Ramis, C., Dolfi, M., Vagenas, P., Omenetti, M., Dinkla, K., Kim, Y., Weber, V., Morin, L., Meijer, I., Kuropiatnyk, V., Strohmeier, T., Gurbuz, A., Staar, P.: Advanced Layout Analysis Models for Docling (2025). <https://doi.org/10.48550/arXiv.2509.11720>
- [73] IBM Research: Docling Concepts: Chunking. Deep Search for Scientific Discovery (DS4SD), (2024). Deep Search for Scientific Discovery (DS4SD)
- [74] Nussbaum, Z., Morris, J.X., Duderstadt, B., Mulyar, A.: Nomic Embed: Training a Reproducible Long Context Text Embedder (2024)
- [75] Muennighoff, N., Tazi, N., Magne, L., Reimers, N.: MTEB: Massive text embedding benchmark. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2023), pp. 2014–2037 (2023). arXiv:2210.07316
- [76] LanceDB: LanceDB: Developer-friendly, serverless vector database. <https://www.lancedb.com/>. Accessed: Apr. 18, 2026 (2024)
- [77] Voorhees, E.M.: The TREC-8 question answering track report. In: Proceedings of the Eighth Text REtrieval Conference (TREC-8), pp. 77–82. NIST Special Publication 500-246, ??? (1999)
- [78] Efron, B.: Better bootstrap confidence intervals. Journal of the American Statistical Association **82**(397), 171–185 (1987)
- [79] Smucker, M.D., Allan, J., Carterette, B.: A comparison of statistical significance tests for information retrieval evaluation. In: Proceedings of the Sixteenth ACM Conference on Conference on Information and Knowledge Management. CIKM '07, pp. 623–632. Association for Computing Machinery, New York, NY, USA (2007). <https://doi.org/10.1145/1321440.1321528> . <https://doi.org/10.1145/1321440.1321528>
- [80] Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., *et al.*: SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods **17**, 261–272 (2020) <https://doi.org/10.1038/s41592-019-0686-2>
- [81] McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947) <https://doi.org/10.1007/BF02295996>
- [82] Dietterich, T.G.: Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation **10**(7), 1895–1923 (1998) <https://doi.org/10.1162/089976698300017197>
- [83] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z.,

- Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. <https://arxiv.org/abs/2306.05685> (Retrieved: 2026-01-24) (2023)
- [84] Rebholz-Schuhmann, D., Jimeno Yepes, A.J., Mulligen, E.M., Kang, N., Kors, J., Milward, D., Corbett, P., Buyko, E., Beisswanger, E., Hahn, U.: Calbc silver standard corpus. *Journal of Bioinformatics and Computational Biology* **08**(01), 163–179 (2010) <https://doi.org/10.1142/S0219720010004562> <https://doi.org/10.1142/S0219720010004562>
- [85] Courty, B., Schmidt, V., Goyal-Kamal, MarionCoutarel, Feld, B., Lecourt, J., LiamConnell, SabAmine, inimaz, supatomic, Léval, M., Blanche, L., Cruveiller, A., ouminasara, Zhao, F., Joshi, A., Bogroff, A., Saboni, A., Lavoreille, H., Laskaris, N., Abati, E., Blank, D., Wang, Z., Catovic, A., alencon, Stechly, M., Bauer, C., Lucas-Otavio, JPW, MinervaBooks: mlco2/codecarbon: v2.4.1. Zenodo (2024). <https://doi.org/10.5281/zenodo.11171501> . <https://doi.org/10.5281/zenodo.11171501>